Statistical Computing

Lecture notes, Lectures 1–11

Contents

Lecture 1. Computer Arithmetic 4

1.1 Storage units and characters 4

1.2 Integers: fixed-point numbers 4

1.3 Floating-point numbers 4

1.4 Rounding and machine epsilon 5

1.5 Floating-point arithmetic and its pitfalls 6

Lecture 2. Linear Systems: LU Decomposition 9

2.1 Algorithms and their cost 9

2.2 Triangular systems 9

2.3 Gaussian elimination and the LU decomposition 9

2.4 Pivoting, conditioning and stability 10

Lecture 3. Cholesky Decomposition 12

3.1 The Cholesky decomposition 12

3.2 Applications 13

3.3 The sweep operator 14

Lecture 4. Least Squares: QR Decomposition 16

4.1 QR decomposition and least squares 16

4.2 Gram–Schmidt 16

4.3 Householder reflections 17

4.4 Givens rotations 18

4.5 Conditioning of least squares 18

4.6 Underdetermined least squares 19

Lecture 5. Iterative Methods for Linear Systems 20

5.1 Direct versus iterative methods 20

5.2 Splitting and convergence 20

5.3 Classical iterations 21

5.4 Conjugate gradient 22

Lecture 6. Nonlinear Equations 24

6.1 Bisection 24

6.2 Fixed-point iteration 24

6.3 Newton's method 25

6.4 Systems of equations 26

Lecture 7. Optimization 27

7.1 Golden section search 27

7.2 Newton's method 27

7.3 Practical Newton: positive definite approximations and line search 27

7.4 Fisher scoring 28

7.5 Generalized linear models 29

7.6 Nonlinear least squares: Gauss–Newton 30

7.7 Gradient descent 30

7.8 Coordinate descent 32

Lecture 8. Numerical Integration 33

8.1 Newton–Cotes rules 33

8.2 Gaussian quadrature 34

8.3 Laplace approximation 37

Lecture 9. Random Number Generation 39

9.1 Uniform random numbers 39

9.2 The inverse CDF method 39

9.3 Normal random numbers 40

9.4 Generating from definitions 41

9.5 Acceptance–rejection sampling 41

9.6 Multivariate distributions 43

Lecture 10. Monte Carlo Integration 44

10.1 Naive Monte Carlo 44

10.2 Importance sampling 44

10.3 Antithetic variates 45

10.4 Control variates 46

10.5 A comparison: the size of a test 46

Lecture 11. Markov Chain Monte Carlo 48

11.1 Markov chains 48

11.2 The Metropolis–Hastings algorithm 49

11.3 The Gibbs sampler 50

11.4 Practical issues 51

Lecture 1. Computer Arithmetic

1.1 Storage units and characters

A **bit** (binary digit, a word coined by Tukey) takes the values 0 and 1; a **byte** is 8 bits. Decimal prefixes (KB = 10^3, MB = 10^6, GB = 10^9, TB = 10^{12} bytes, ...) are distinguished from binary ones (KiB = 2^{10}, MiB = 2^{20} bytes, ...).

Text is stored as character codes. ASCII uses 7 bits (128 characters) and extended ASCII 8 bits (256). Unicode encodings (UTF-8, UTF-16, UTF-32) cover all scripts; UTF-8, whose one-byte codes agree with ASCII, is dominant on the internet.

The basic lesson of this lecture: memory is finite, so *every* representation of $\mathbb{Z}$ or $\mathbb{R}$ in a computer is an approximation, and its imperfections must be understood.

1.2 Integers: fixed-point numbers

A fixed-point system with M bits models integers.

- **Unsigned** integers represent $\{0, 1, \ldots, 2^M - 1\}$ (e.g. 0–255 for 8 bits).
- **Signed** integers use the most significant bit as a sign bit and **two's complement** for negatives: to represent $-x$, flip all bits of x and add 1. Equivalently, the bit pattern of $-x$ is that of the unsigned integer $2^M - x$. The range is $[-2^{M-1}, 2^{M-1} - 1]$; for $M = 32$ the maximum is $2^{31} - 1 = 2147483647$.

Two's complement makes subtraction a special case of addition: adding the bit patterns modulo 2^M gives the right answer whenever the true result is in range. For example, with 8 bits, $57 - 35$: $57 = 00111001_2$ and $-35 = 11011101_2$, and the sum modulo 2^8 is $00010110_2 = 22$.

When the result leaves the range we get **overflow**. Languages differ: C and Julia wrap around (modular arithmetic, so $255 + 1 = 0$ for unsigned 8-bit integers), while R returns `NA`. Arbitrary-precision ("big") integers avoid overflow at the cost of speed.

1.3 Floating-point numbers

A real number is written in scientific notation with base $b = 2$:

$$\pm d_1.d_2 \cdots d_p \times 2^e, \qquad d_i \in \{0, 1\}.$$

In **normalized** form $d_1 = 1$ (e.g. $18 = 1.0010_2 \times 2^4$), so the leading bit need not be stored. A number that can be represented exactly is a **floating-point number**; every other real number is represented by a nearby floating-point number. The IEEE 754 standard (1985) stores

- a sign bit,
- the fraction (mantissa, significand) of the normalized representation, $p - 1$ bits,
- the exponent plus a **bias**.

format	bits	fraction bits	p	$e_{\min}, e_{\max}$	range	decimal digits
half (Float16)	16	10	11	$-14, 15$	$10^{\pm 4}$	3.3
single (float)	32	23	24	$-126, 127$	$10^{\pm 38}$	7.2
double	64	52	53	$-1022, 1023$	$10^{\pm 308}$	15.9

The bias is 15, 127 and 1023 respectively; the precision in decimal digits is $\log_{10} 2^p$. For half precision, the value of the bit string $b_{15}b_{14}\cdots b_0$ is

$$(-1)^{b_{15}} \times 2^{(b_{14}\cdots b_{10})_2 - 15} \times \Big(1 + \sum_{i=1}^{10} b_{10-i} 2^{-i}\Big).$$

Double precision is the default for numerical computing (R has no native single precision).

Special values. The exponent codes $e_{\min} - 1$ (all zeros) and $e_{\max} + 1$ (all ones) are reserved.

- Exponent $e_{\max} + 1$ with zero fraction: $\pm\infty$ (e.g. $10^{300} \times 10^{300}$).
- Exponent $e_{\max}+1$ with nonzero fraction: `NaN` (e.g. $0/0$, $0\cdot\infty$). A `NaN` is not equal to anything, including itself.
- Exponent $e_{\min} - 1$ with zero fraction: an exact zero (which cannot be written in normalized form).
- Exponent $e_{\min} - 1$ with nonzero fraction: **denormalized** (subnormal) numbers $0.d_2 \cdots d_p \times 2^{e_{\min}}$, filling the gap $(0, 2^{e_{\min}})$. This **gradual underflow** guarantees that $x - y = 0$ only if $x = y$. For example, in single precision $2^{-149} = 0.0\cdots01_2 \times 2^{-126}$ is representable.

1.4 Rounding and machine epsilon

A real number with more than p significant bits must be rounded. IEEE 754's default is **round to nearest, ties to even**: choose the nearest representable number, and if the number lies exactly midway, choose the one whose last bit is 0. With $p = 4$:

exact	rounded	reason
1.000011×2^1	1.000×2^1	remainder $011 < 100$: down
1.000110×2^1	1.001×2^1	remainder $110 > 100$: up
1.011100×2^1	1.100×2^1	tie: to even (up)
1.010100×2^1	1.010×2^1	tie: to even (down)

If x is represented by $[x]$, the **absolute error** is $|[x] - x|$ and the **relative error** is $|[x] - x|/|x|$.

Floating-point numbers are not evenly spaced: each interval $(2^i, 2^{i+1}]$ contains the same number (2^{p-1}) of them, uniformly spaced, so the spacing doubles from one binade to the next.

Definition (Machine epsilons)

$\epsilon_{\max}$ is the smallest positive floating-point number whose addition to 1 gives a result different from 1; $\epsilon_{\min}$ is the smallest positive floating-point number whose subtraction from 1 gives a result different from 1.

Proposition 1.1

With precision p and ties to even,

$$\epsilon_{\max} = 2^{-p} + 2^{-(2p-1)}, \qquad \epsilon_{\min} = 2^{-(p+1)} + 2^{-2p}.$$

For double precision, $\epsilon_{\max} = 2^{-53} + 2^{-105} \approx 1.1 \times 10^{-16}$.

Proof. The floating-point neighbours of 1 are $1 + 2^{-(p-1)}$ above and $1 - 2^{-p}$ below. The real number $1 + 2^{-p}$ is exactly midway between 1 and $1 + 2^{-(p-1)}$, and the tie goes to the even neighbour 1.

So $\epsilon_{\max} > 2^{-p}$. The next floating-point number after $2^{-p} = 1.0\cdots0_2 \times 2^{-p}$ is $1.0\cdots01_2 \times 2^{-p} = 2^{-p} + 2^{-p-(p-1)}$, and adding it to 1 gives a number past the midpoint, which rounds to $1 + 2^{-(p-1)}$. Hence $\epsilon_{\max} = 2^{-p} + 2^{-(2p-1)}$. Similarly, $1 - 2^{-(p+1)}$ is midway between $1 - 2^{-p}$ and 1 and rounds to the even value 1; the next floating-point number after $2^{-(p+1)}$ is $2^{-(p+1)} + 2^{-(p+1)-(p-1)} = 2^{-(p+1)} + 2^{-2p}$, which does move 1. ■

Theorem 1.2 (Relative error of representation)

If a real number $x > 0$ in the normal range is represented by $[x]$, then

$$\frac{|[x] - x|}{|x|} \le \epsilon_{\max}.$$

Proof. Let $[x] = (1 + \sum_{i=1}^{p-1} b_{i+1} 2^{-i}) \times 2^e$, so $[x] \ge 2^e$ and the spacing of floating-point numbers near $[x]$ is $2^{e-(p-1)}$. Rounding to nearest gives $|x - [x]| \le 2^e/2^p$. Then

$$\frac{|x - [x]|}{|x|} \le \frac{2^e}{2^p} \cdot \frac{1}{[x] - 2^e/2^p} \le \frac{2^e}{2^p} \cdot \frac{1}{2^e(1 - 2^{-p})} \le \frac{1}{2^p}\left(1 + \frac{1}{2^{p-1}}\right) = \epsilon_{\max},$$

using $1/(1-u) \le 1 + 2u$ for $0 \le u \le \frac{1}{2}$. ■

So $[x] = x(1+\delta)$ with $|\delta| \le \epsilon_{\max}$: the machine epsilon is the **unit roundoff** or machine precision.

Example 1.1 (A toy decimal system)

Take $b = 10$, $p = 5$, $-9 \le e \le 9$, numbers $\pm 0.d_1\cdots d_5 \times 10^e$, normalized except that subnormals and zero are allowed at $e = -9$. Let $X \sim \mathrm{Unif}(0,1)$ and round to nearest.

- The smallest positive number is $0.00001 \times 10^{-9} = 10^{-14}$, so $[X] = 0$ iff $X \le \frac{1}{2} \cdot 10^{-14}$ (ties to the even neighbour 0): $P([X] = 0) = 5 \times 10^{-15}$.
- $1 = 0.10000 \times 10^1$ and its predecessor is 0.99999×10^0, so $[X] = 1$ iff $X \ge 0.999995$: $P([X] = 1) = 5 \times 10^{-6}$.
- $[X]$ is not uniform over the floating-point numbers in $(0,1)$: within $[10^{e-1}, 10^e)$ the spacing is 10^{e-5}, so each number there has probability 10^{e-5}. For instance $P([X] = 0.5) = 10^{-5}$ but $P([X] = 0.05) = 10^{-6}$.

1.5 Floating-point arithmetic and its pitfalls

IEEE 754 requires that $+, -, \times, \div, \sqrt{\ }$, remainder and conversions be computed *as if exactly* and then rounded: the computed sum of floating-point numbers $[x]$ and $[y]$ is $[[x] + [y]]$.

Overflow and underflow. Results beyond $\approx 10^{308}$ in magnitude overflow to $\pm\infty$; results below $\approx 10^{-308}$ underflow gradually to subnormals and then 0. Underflow is usually the lesser evil, so formulas should be arranged to avoid overflow. For instance, the logistic function should be evaluated as

$$p = \frac{1}{1 + e^{-x^\top \beta}} \qquad \text{rather than} \qquad \frac{e^{x^\top \beta}}{1 + e^{x^\top \beta}},$$

since the latter gives $\infty/\infty =$ `NaN` for large $x^\top\beta$, while the former only underflows gracefully. Similarly $\log(e^{709})$ is fine but $\log(e^{710}) = \infty$; computations with likelihoods should be done on the log scale.

Adding numbers of different magnitudes. If $a = x.xx\cdots \times 2^{30}$ and $b = y.yy\cdots \times 2^{-30}$, then b falls below the last bit of a and $a + b = a$ in double precision: the information in b is lost.

Proposition 1.3 (Error of a sum)

For $x, y > 0$, the computed sum satisfies

$$\frac{|[[x] + [y]] - (x + y)|}{x + y} \leq 2\epsilon_{\max} + \epsilon_{\max}^2 \approx 2\epsilon_{\max}.$$

Proof. Write $[x] = x(1+\delta_x)$, $[y] = y(1+\delta_y)$ and $[[x]+[y]] = ([x]+[y])(1+\delta_s)$, all $|\delta| \leq \epsilon_{\max}$. Then $[x] + [y] \leq (x + y)(1 + \epsilon_{\max})$ and

$$|[[x] + [y]] - (x + y)| \leq |\delta_s|([x] + [y]) + |\delta_x|x + |\delta_y|y \leq \epsilon_{\max}(1 + \epsilon_{\max})(x + y) + \epsilon_{\max}(x + y).$$

■

So adding positive numbers is benign. Subtraction is not.

Catastrophic cancellation. Subtracting two nearly equal numbers that carry rounding errors destroys significant digits. With 3 decimal digits, $b = 3.34$, $a = 1.22$, $c = 2.28$: the exact discriminant is $b^2 - 4ac = 0.0292$, but $b^2 = 11.2$ and $4ac = 11.1$ after rounding, giving 0.1. The subtraction itself was exact; the errors were introduced earlier and then magnified. In binary: if $[x] = 1.d_2 \cdots d_{p-1}1$ and $[y] = 1.d_2 \cdots d_{p-1}0$, then $[x] - [y] = 2^{-(p-1)}$ exactly, while the true $x - y$ may be anywhere in $(0, 2^{-(p-2)} + 2^{-2p}]$. The relative error is unbounded: no significant digit is guaranteed. Likewise $1 - \Phi(z)$ computed as one minus the cdf is useless for large z (it returns 0 for $z \geq 9$), and tail probabilities must be computed directly.

Rules of thumb.

(1) Add small numbers before large ones.

(2) Add numbers of like magnitude together (pairwise summation).

(3) Avoid subtracting nearly equal numbers. For example, for $ax^2 + bx + c = 0$ with $b > 0$, compute $x_1 = \frac{-b-\sqrt{b^2-4ac}}{2a}$ (no cancellation) and then $x_2 = c/(ax_1)$ from $x_1x_2 = c/a$, instead of $\frac{-b+\sqrt{b^2-4ac}}{2a}$.

Floating-point arithmetic also violates familiar algebraic laws: addition is not associative ($(a+b)-a \neq b$ when $a \gg b$) and multiplication does not distribute over addition exactly. Even the additive identity misbehaves: zero has two representations ± 0 (equal under comparison), $x + 0 = x$ fails for $x =$ `NaN` (`NaN` equals nothing), and $x - x = 0$ fails for $x = \pm\infty$ and `NaN`. Familiar decimal constants are also inexact: 0.1, 0.3 have infinite binary expansions, so `0.1+0.2==0.3` and `0.1==0.3/3` are both false; equality of computed reals should be tested with a tolerance (R's `all.equal`).

Example 1.2 (Cancellation in the quadratic formula)

For $a = 10^{-30}$, $b = -10^{30}$, $c = 10^{30}$ the roots are about 10^{60} and 1. In double precision $4ac = 4$ is far below the last bit of $b^2 = 10^{60}$, so the computed discriminant equals b^2 and $\sqrt{b^2 - 4ac} = |b|$ exactly. The textbook formula then returns $(-b - |b|)/(2a) = 0$ for the small root: every digit is lost to cancellation. The stable recipe is

$$q = -\tfrac{1}{2}(b + \operatorname{sign}(b)\sqrt{b^2 - 4ac}), \qquad x_1 = q/a, \qquad x_2 = c/q,$$

which never subtracts nearly equal numbers and gives $x_2 = 1$ to full precision. (With $b =$

$+10^{30}$ the same happens with the signs reversed.)

Example 1.3 (The harmonic series "converges")

Summing $1/i$ in a loop, the partial sums $s_n \approx \log n$ diverge mathematically, but on a computer they stop changing: once $1/n$ is below half the spacing of floating-point numbers near s_n, the addition $s_n + 1/n$ rounds back to s_n. In single precision this happens at $n = 2^{21} = 2097152$ with $s = 15.4037$; in double precision the spacing near $s \in [32, 64)$ is 2^{-47}, so the sum freezes near 34 after about $2^{48} \approx 3 \times 10^{14}$ terms.

The same series gives Euler's constant $\gamma = \lim_n(\sum_{i \le n} 1/i - \log n) \approx 0.5772$, i.e. $f(n) = \log n$ in $\sum_{i \le n} 1/i = f(n) + \gamma + o(1)$. To compute it accurately: choose a moderate n (say 10^6), add the terms in *increasing* order $1/n + \cdots + 1/1$ (small numbers first), subtract $\log n$, and remove the leading error term, which is $1/(2n)$. With $n = 10^6$ this gives γ to about 13 digits, while the uncorrected difference is off by 5×10^{-7}.

Lecture 2. Linear Systems: LU Decomposition

2.1 Algorithms and their cost

We study algorithms for solving $Ax = b$, a ubiquitous task in statistics. A guiding principle (Gentle):

Mantra

The form of a mathematical expression and the way the expression should be evaluated in practice may be quite different.

An **algorithm** is a finite set of instructions mapping an input to an output. Its cost is counted in **flops**: one floating-point addition, subtraction, multiplication, division or comparison (with the accompanying fetch and store). An algorithm has order $O(f(n))$ if its leading flop count is $c\,f(n)$. For example, Ab with $A \in \mathbb{R}^{m\times n}$ costs $2mn$ flops and AB with $B \in \mathbb{R}^{n\times p}$ costs $2mnp$.

Orders matter enormously at scale: on a machine doing 10^{12} flops per second, a problem of size $n = 10^{12}$ takes about 27 seconds with an $O(n \log n)$ algorithm but about 31700 years with an $O(n^2)$ one. Quicksort and Tukey's fast Fourier transform are celebrated examples that turn $O(n^2)$ into $O(n \log n)$.

2.2 Triangular systems

The strategy for general systems is to reduce them to easy ones. If A is lower triangular, **forward substitution** solves $Ax = b$:

$$x_1 = \frac{b_1}{a_{11}}, \qquad x_i = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j\Big), \quad i = 2, \dots, n,$$

and if A is upper triangular, **back substitution** solves from x_n upwards. Each costs $1 + 3 + 5 + \cdots + (2n-1) = n^2$ flops, and can be done in place (b overwritten by x). In BLAS this is the routine `trsm`.

Useful facts about triangular matrices: the eigenvalues are the diagonal entries; the determinant is their product; products and inverses of (unit) lower triangular matrices are (unit) lower triangular, and similarly for upper triangular. For products, $(AB)_{ij} = \sum_k a_{ik}b_{kj}$ with A, B upper triangular needs $i \le k \le j$, so it vanishes for $i > j$ and equals $a_{ii}b_{ii}$ for $i = j$. For inverses, induct on n with $A = \left(\begin{smallmatrix} A_{11} & a_{12} \\ 0 & a_{22} \end{smallmatrix}\right)$: the bottom-left block B_{21} of $B = A^{-1}$ satisfies $a_{22}B_{21} = 0$, so $B_{21} = 0$ and $A_{11}B_{11} = I$, and $B_{11} = A_{11}^{-1}$ is upper triangular by induction (with unit diagonal if A has one). The lower triangular cases follow by transposition. Consequently an orthogonal triangular matrix Q is diagonal: Q^{-1} is upper triangular while $Q^\top$ is lower triangular, and they are equal.

2.3 Gaussian elimination and the LU decomposition

The **elementary operator matrix** $E_{jk}(c) = I + c\,e_j e_k^\top$ $(j \ne k)$ is unit triangular with $E_{jk}(c)^{-1} = E_{jk}(-c)$, and left-multiplying a matrix by it adds c times row k to row j ($2m$ flops for m columns). Gaussian elimination applies a sequence of such operations to reduce A to upper triangular form.

Combining the operations on column k gives the **Gauss transformation**

$$M_k = E_{nk}(c_{nk}) \cdots E_{k+1,k}(c_{k+1,k}) = I + \mathbf{c}_k e_k^\top, \qquad \mathbf{c}_k = (0, \dots, 0, c_{k+1,k}, \dots, c_{nk})^\top,$$

with $c_{ik} = -a_{ik}^{(k)}/a_{kk}^{(k)}$, which zeros column k below the diagonal. It is unit lower triangular and $M_k^{-1} = I - \mathbf{c}_k e_k^\top$. Elimination computes

$$M_{n-1}\cdots M_1 A = U \quad \text{(upper triangular)}, \qquad \text{so} \qquad A = LU, \quad L = M_1^{-1}\cdots M_{n-1}^{-1}.$$

Proposition 2.1

$L = I - \sum_{k=1}^{n-1} \mathbf{c}_k e_k^\top$: it is unit lower triangular, and its (i,k) entry below the diagonal is the multiplier $\ell_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)}$.

Proof. $M_k^{-1}M_{k+1}^{-1} = (I - \mathbf{c}_k e_k^\top)(I - \mathbf{c}_{k+1}e_{k+1}^\top) = I - \mathbf{c}_k e_k^\top - \mathbf{c}_{k+1}e_{k+1}^\top + \mathbf{c}_k(e_k^\top \mathbf{c}_{k+1})e_{k+1}^\top$, and $e_k^\top \mathbf{c}_{k+1} = 0$ because $\mathbf{c}_{k+1}$ vanishes in positions $\le k+1$. Inductively the product is $I - \sum_k \mathbf{c}_k e_k^\top$. ■

Theorem 2.2 (Existence and uniqueness of LU)

If the leading principal submatrices $A[1{:}k, 1{:}k]$, $k = 1, \ldots, n-1$, are nonsingular, then $A = LU$ with L unit lower triangular and U upper triangular. If moreover A is nonsingular, the decomposition is unique and $\det A = \prod_k u_{kk}$.

Proof. *Existence.* The pivot at step k is $a_{kk}^{(k)} = \det A[1{:}k, 1{:}k]/\det A[1{:}k-1, 1{:}k-1]$, because elimination with unit lower triangular operations preserves leading principal minors. So all pivots are nonzero and elimination runs to completion. *Uniqueness.* If $L_1U_1 = L_2U_2$ with A nonsingular, then all factors are invertible and $L_2^{-1}L_1 = U_2U_1^{-1}$. The left side is unit lower triangular and the right side upper triangular, so both equal I. Finally $\det A = \det L \det U = \prod u_{kk}$. (This is how software computes determinants.) ■

For nonsingular A the condition is also necessary, so a nonsingular matrix has an LU decomposition (without pivoting) *if and only if* all its leading principal submatrices are nonsingular. Indeed, partitioning $A = LU$ conformally with the leading $k \times k$ block gives $A[1{:}k, 1{:}k] = L_kU_k$ (the zero blocks of L and U kill the other terms), where L_k, U_k are the leading blocks of L, U. Since $0 \neq \det A = \prod_i u_{ii}$, all $u_{ii} \neq 0$, hence $\det A[1{:}k, 1{:}k] = \prod_{i \le k} u_{ii} \neq 0$.

Cost. Step k updates an $(n-k) \times (n-k)$ block at 2 flops per entry, so LU costs

$$\sum_{k=1}^{n-1} 2(n-k)^2 \approx \tfrac{2}{3}n^3 \text{ flops},$$

done in place (A overwritten by L below the diagonal and U on and above it). Solving $LUx = b$ then costs $2n^2$ (one forward and one back substitution), $\frac{2}{3}n^3 + 2n^2$ in total. With several right-hand sides the factorization is done once.

No inversion. Inverting A via LU solves n systems with right-hand sides $e_1, \ldots, e_n$; exploiting the zeros, this costs $\frac{4}{3}n^3$, so $2n^3$ in total, three times the cost of one solve. Hence the rule: *whenever we see $A^{-1}b$, solve $Ax = b$ instead.* Explicit inverses are needed only for standard errors, for very many right-hand sides, or for small n.

2.4 Pivoting, conditioning and stability

For $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $Ax = b$ is trivially solvable but elimination fails at once (zero pivot). Small pivots are just as harmful.

Example 2.1

Solve $0.001x_1+x_2=1$, $x_1+x_2=2$ (exact solution $\approx(1.001, 0.999)$) with two significant digits. Eliminating with the pivot 0.001 gives multiplier 1000 and $(1-1000)x_2 = 2-1000$, rounded to $-1000x_2 = -1000$; so $x_2 = 1$ and then $x_1 = 0$. The computed factors $L = \begin{pmatrix} 1 & 0 \\ 1000 & 1 \end{pmatrix}$, $U = \begin{pmatrix} 0.001 & 1 \\ 0 & -1000 \end{pmatrix}$ multiply to $\begin{pmatrix} 0.001 & 1 \\ 1 & 0 \end{pmatrix} \neq A$: the large multiplier wiped out a_{22} and b_2, as in catastrophic cancellation. Swapping the equations first gives multiplier 0.001, $(1-0.001)x_2 = 1-0.002$, i.e. $x_2 = 1$, $x_1 = 1$: correct to two digits.

Conditioning. The *problem* $A \mapsto A^{-1}b$ is **well conditioned** if small relative perturbations of A cause small relative changes of the solution. It is measured by the **condition number**

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}, \qquad \sigma_{\max}(A) = \max_{x\neq 0} \frac{\|Ax\|}{\|x\|}, \quad \sigma_{\min}(A) = \min_{x\neq 0} \frac{\|Ax\|}{\|x\|},$$

which is ∞ for singular A. In Example 2.1, $\kappa(A) = 2.62$: the problem is well conditioned.

Example 2.2 (An ill-conditioned matrix)

The symmetric matrix

$$A = \begin{pmatrix} 5 & 7 & 6 & 5 \\ 7 & 10 & 8 & 7 \\ 6 & 8 & 10 & 9 \\ 5 & 7 & 9 & 10 \end{pmatrix}, \qquad A^{-1} = \begin{pmatrix} 68 & -41 & -17 & 10 \\ -41 & 25 & 10 & -6 \\ -17 & 10 & 5 & -3 \\ 10 & -6 & -3 & 2 \end{pmatrix},$$

looks harmless. Change a_{33} from 10 to 9.99, a relative change of 10^{-3} in one entry. The inverse becomes (to two decimals) $\begin{pmatrix} 71.04 & -42.79 & -17.89 & 10.54 \\ -42.79 & 26.05 & 10.53 & -6.32 \\ -17.89 & 10.53 & 5.26 & -3.16 \\ 10.54 & -6.32 & -3.16 & 2.09 \end{pmatrix}$, whose entries moved by up to 5.4%. Measuring condition by the ratio of maximal relative entrywise changes, the perturbation was amplified about 54 times, so $\kappa(A)$ is at least of that order. The amplification depends on the direction of the perturbation; the worst case is much larger: $\kappa_2(A) = \sigma_{\max}/\sigma_{\min} \approx 2984$, so about 3.5 decimal digits can be lost in solving $Ax = b$.

Stability. An *algorithm* $\tilde{f}$ is **stable** if for every input $\tilde{A}$ within relative distance ϵ of A, the computed $\hat{x} = \tilde{f}(A)$ is within relative distance $O(\epsilon)$ of $\tilde{A}^{-1}b$. Example 2.1 shows that elimination without pivoting is unstable: a well-conditioned problem gives relative error $\|x-\hat{x}\|/\|x\| \geq 1/\sqrt{2}$.

Partial pivoting. Before eliminating column k, swap into row k the row with the largest $|a_{ik}|$, $i \geq k$. This costs only $O(n^2)$ comparisons and gives

$$PA = LU, \qquad P \text{ a permutation matrix}, \quad |\ell_{ij}| \leq 1.$$

Complete pivoting also swaps columns, moving the largest entry of the remaining submatrix to the pivot position: $PAQ = LU$. It is more stable but costs more. Gaussian elimination with partial pivoting (GEPP) is the standard method for general dense systems and is stable in practice, partly because the multipliers are bounded by 1.

Software. Dense linear algebra rests on BLAS (basic operations) and LAPACK (decompositions). The LAPACK routine `getrf` computes $PA = LU$ in place; R's `solve` calls `dgesv`, which factors and solves.

Lecture 3. Cholesky Decomposition

Mantra 2

The structure should be exploited whenever possible in solving a problem.

Common structures are symmetry, positive (semi)definiteness, sparsity, Kronecker products and low rank. Statisticians mostly meet positive (semi)definite matrices: covariance matrices and Gram matrices $X^\top X$. A matrix M is positive definite (semidefinite) if $x^\top Mx > 0$ (≥ 0) for all $x \neq 0$; since $x^\top Mx = x^\top \frac{M+M^\top}{2} x$, we may assume M symmetric.

3.1 The Cholesky decomposition

Motivation. Statistical computations constantly solve systems $A\mathbf{x} = \mathbf{b}$ with A symmetric positive definite (normal equations, Gaussian likelihoods). Cholesky factors such A as $LL^\top$ at half the cost of LU, with no pivoting and excellent stability.

Theorem 3.1 (Cholesky)

If $A \in \mathbb{R}^{n\times n}$ is symmetric positive definite, then $A = LL^\top$ for a unique lower triangular L with positive diagonal entries.

Proof. Induction on n. For $n = 1$, $a = \sqrt{a}\sqrt{a}$ with $a > 0$. For $n > 1$, partition and match blocks:

$$\begin{pmatrix} a_{11} & \mathbf{a}^\top \\ \mathbf{a} & A_{22} \end{pmatrix} = \begin{pmatrix} \ell_{11} & \mathbf{0}^\top \\ \mathbf{b} & L_{22} \end{pmatrix} \begin{pmatrix} \ell_{11} & \mathbf{b}^\top \\ \mathbf{0} & L_{22}^\top \end{pmatrix} \iff a_{11} = \ell_{11}^2, \quad \mathbf{a} = \ell_{11}\mathbf{b}, \quad A_{22} = \mathbf{b}\mathbf{b}^\top + L_{22}L_{22}^\top.$$

Since $a_{11} = e_1^\top Ae_1 > 0$, the conditions force $\ell_{11} = \sqrt{a_{11}}$ and $\mathbf{b} = \mathbf{a}/\sqrt{a_{11}}$. (Every principal submatrix of a positive definite matrix is positive definite, e.g. $\mathbf{y}^\top A_{22}\mathbf{y} = (0, \mathbf{y}^\top)A(0, \mathbf{y}^\top)^\top > 0$; but the recursion needs more.) The **Schur complement** $S = A_{22} - \mathbf{b}\mathbf{b}^\top = A_{22} - \mathbf{a}\mathbf{a}^\top/a_{11}$ is positive definite: for $\mathbf{y} \neq \mathbf{0}$, put $x = (-\mathbf{a}^\top\mathbf{y}/a_{11}, \mathbf{y}^\top)^\top \neq 0$; then

$$0 < x^\top Ax = \frac{(\mathbf{a}^\top\mathbf{y})^2}{a_{11}} - 2\frac{(\mathbf{a}^\top\mathbf{y})^2}{a_{11}} + \mathbf{y}^\top A_{22}\mathbf{y} = \mathbf{y}^\top S\mathbf{y}.$$

By the induction hypothesis $S = L_{22}L_{22}^\top$ for a unique such L_{22}, which completes existence and uniqueness. ■

The proof is the algorithm: compute the first column of L, update the Schur complement, and recurse; this can be done in place in the lower triangle of A.

Cost. The Schur complement updates are symmetric, so only half the entries are computed:

$$\tfrac{1}{2}\{2(n-1)^2 + 2(n-2)^2 + \cdots + 2\} \approx \tfrac{1}{3}n^3 \text{ flops}$$

plus n square roots, *half* the cost of LU. The algorithm is very stable without pivoting, and it fails (a nonpositive number under a square root) exactly when A is not positive definite. So Cholesky is the cheapest way to *test* positive definiteness.

Example 3.1

For

$$A = \begin{pmatrix} 4 & 6 & 4 & 4 \\ 6 & 37/4 & 11/2 & 6 \\ 4 & 11/2 & 29/4 & 7 \\ 4 & 6 & 7 & 33/4 \end{pmatrix}$$

the first column of L is $(4,6,4,4)^\top/2 = (2,3,2,2)^\top$. The Schur complement $A_{22} - \mathbf{b}\mathbf{b}^\top$ has first column $(37/4 - 9,\ 11/2 - 6,\ 6 - 6)^\top = (1/4, -1/2, 0)^\top$, so $\ell_{22} = 1/2$ and the second column of L is $(1/2, -1, 0)^\top$. Continuing, the pivots are $4, \frac{1}{4}, \frac{9}{4}, \frac{1}{4}$, all positive, and

$$L = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 3 & 1/2 & 0 & 0 \\ 2 & -1 & 3/2 & 0 \\ 2 & 0 & 2 & 1/2 \end{pmatrix}.$$

Since the algorithm ran to completion with positive pivots, $A = LL^\top$ is positive definite (and $\det A = (2 \cdot \frac{1}{2} \cdot \frac{3}{2} \cdot \frac{1}{2})^2 = \frac{9}{16}$).

Pivoting for semidefinite matrices. If A is only positive semidefinite (e.g. $X^\top X$ with X rank deficient), a zero pivot can occur. With **symmetric pivoting**, at stage k the largest remaining diagonal entry is moved (by a symmetric row and column swap) to the pivot position. If the largest remaining diagonal entry is 0, the whole remaining block is 0: a positive semidefinite matrix with zero diagonal entry a_{ii} has zero row i, since $|a_{ij}| \le \sqrt{a_{ii}a_{jj}}$. The algorithm then stops, giving

$$PAP^\top = LL^\top, \qquad L \in \mathbb{R}^{n \times r}, \quad r = \operatorname{rank} A.$$

LAPACK provides `potrf` (no pivoting) and `pstrf` (pivoting); R's `chol` returns the upper factor $R = L^\top$, $A = R^\top R$.

3.2 Applications

Multivariate normal log-likelihood. For $y \sim N_n(0, \Sigma)$,

$$\log f(y) = -\frac{n}{2}\log(2\pi) - \frac{1}{2}\log\det\Sigma - \frac{1}{2}y^\top\Sigma^{-1}y.$$

- Method 1: invert Σ ($2n^3$), form the quadratic form ($2n^2$), compute the determinant by LU ($\frac{2}{3}n^3$).
- Method 2: factor $\Sigma = LL^\top$ ($\frac{1}{3}n^3$), solve $Lx = y$ by forward substitution (n^2), then $y^\top\Sigma^{-1}y = x^\top x$ ($2n$) and $\log\det\Sigma = 2\sum_i \log \ell_{ii}$ (n).

Method 2 is about 8 times cheaper and more accurate, and the log-determinant computed this way cannot overflow. To evaluate the density at many points, factor once and pay n^2 per point.

Linear regression. For $X \in \mathbb{R}^{n \times p}$, solve the normal equations $X^\top X\beta = X^\top y$ by Cholesky:

form $X^\top X$	np^2 (symmetric)
form $X^\top y$	$2np$
Cholesky of $X^\top X$	$\frac{1}{3}p^3$
two triangular solves	$2p^2$
$(X^\top X)^{-1}$ for standard errors	$\frac{4}{3}p^3$

In total about $np^2 + \frac{1}{3}p^3$ flops, or $np^2 + \frac{5}{3}p^3$ with standard errors. This is the fastest method when $n \gg p$, but forming $X^\top X$ squares the condition number (Lecture 4).

3.3 The sweep operator

For a symmetric $A = (a_{ij}) \in \mathbb{R}^{n\times n}$ with $a_{kk} \neq 0$, **sweeping** on the kth diagonal entry maps A to $\hat{A} = (\hat{a}_{ij})$ with

$$\hat{a}_{kk} = -\frac{1}{a_{kk}}, \qquad \hat{a}_{ik} = \frac{a_{ik}}{a_{kk}}, \qquad \hat{a}_{kj} = \frac{a_{kj}}{a_{kk}}, \qquad \hat{a}_{ij} = a_{ij} - \frac{a_{ik}a_{kj}}{a_{kk}} \quad (i, j \neq k).$$

The last formula is one step of Gaussian elimination (a Schur complement update), so a sweep costs about n^2 flops ($n^2/2$ using symmetry), and it can be done in place. The **inverse sweep** on entry k is

$$\check{a}_{kk} = -\frac{1}{a_{kk}}, \qquad \check{a}_{ik} = -\frac{a_{ik}}{a_{kk}}, \qquad \check{a}_{kj} = -\frac{a_{kj}}{a_{kk}}, \qquad \check{a}_{ij} = a_{ij} - \frac{a_{ik}a_{kj}}{a_{kk}};$$

substituting $\hat{A}$ shows directly that it restores A (e.g. $\hat{a}_{ij} - \hat{a}_{ik}\hat{a}_{kj}/\hat{a}_{kk} = a_{ij} - a_{ik}a_{kj}/a_{kk} + a_{ik}a_{kj}/a_{kk} = a_{ij}$).

Lemma 3.2 (Exchange property)

Let $U = [u_1, \ldots, u_n]$ and $V = [v_1, \ldots, v_n]$ be $p \times n$ with $V = UA$. After sweeping on entry k,

$$[v_1, \ldots, v_{k-1}, -u_k, v_{k+1}, \ldots, v_n] = [u_1, \ldots, u_{k-1}, v_k, u_{k+1}, \ldots, u_n]\,\hat{A}.$$

That is, sweeping exchanges the roles of u_k and v_k (up to sign).

Proof. Column k of the right side is $-v_k/a_{kk} + \sum_{i\neq k} u_i a_{ik}/a_{kk} = -(v_k - \sum_{i\neq k} u_i a_{ik})/a_{kk} = -u_k a_{kk}/a_{kk} = -u_k$, using $v_k = \sum_i u_i a_{ik}$. For $j \neq k$, column j is

$$v_k \frac{a_{kj}}{a_{kk}} + \sum_{i\neq k} u_i \Big(a_{ij} - \frac{a_{ik}a_{kj}}{a_{kk}}\Big) = \sum_{i\neq k} u_i a_{ij} + \frac{a_{kj}}{a_{kk}}\Big(v_k - \sum_{i\neq k} u_i a_{ik}\Big) = \sum_{i\neq k} u_i a_{ij} + a_{kj}u_k = v_j.$$

■

Proposition 3.3

Partition $A = \left(\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right)$ with $A_{11} \in \mathbb{R}^{k\times k}$ invertible. Sweeping on the diagonal entries $1, \ldots, k$ (in any order, as long as each pivot is nonzero) gives

$$\hat{A} = \begin{pmatrix} -A_{11}^{-1} & A_{11}^{-1}A_{12} \\ A_{21}A_{11}^{-1} & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{pmatrix}.$$

Proof. Apply the lemma to $A = IA$ ($U = I$, $V = A$) for each sweep. After sweeping $1, \ldots, k$,

$$\begin{pmatrix} -I_k & A_{12} \\ 0 & A_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & 0 \\ A_{21} & I \end{pmatrix} \begin{pmatrix} \hat{A}_{11} & \hat{A}_{12} \\ \hat{A}_{21} & \hat{A}_{22} \end{pmatrix}.$$

Reading off blocks: $A_{11}\hat{A}_{11} = -I$, $A_{11}\hat{A}_{12} = A_{12}$, $A_{21}\hat{A}_{11} + \hat{A}_{21} = 0$ and $A_{21}\hat{A}_{12} + \hat{A}_{22} = A_{22}$, which give the four blocks. The result depends only on the set of swept indices, so sweeps commute. ■

Regression by sweeping. Sweep the $(p+1)\times(p+1)$ matrix $\left(\begin{smallmatrix} X^\top X & X^\top y \\ y^\top X & y^\top y \end{smallmatrix}\right)$ on its first p diagonal entries. By the proposition the result is

$$\begin{pmatrix} -(X^\top X)^{-1} & (X^\top X)^{-1}X^\top y \\ y^\top X(X^\top X)^{-1} & y^\top y - y^\top X(X^\top X)^{-1}X^\top y \end{pmatrix} = \begin{pmatrix} -\sigma^{-2}\operatorname{Cov}(\hat\beta) & \hat\beta \\ \hat\beta^\top & \|y-\hat y\|^2 \end{pmatrix}:$$

the coefficients, their (unscaled) covariance and the residual sum of squares appear at once. This makes sweeping ideal for **stepwise regression**. Adding variable j to the current model is one sweep on entry j and removing it is one inverse sweep, each $O(p^2)$, instead of refitting from scratch; the swept matrix always holds the current $\hat\beta$, standard errors and RSS, from which the F (or $t^2=\hat\beta_j^2/\widehat{\operatorname{Var}}\hat\beta_j$) statistics for entering or leaving variables are read off. A pivot that is (numerically) zero signals a variable that is collinear with those already in the model. The price is that it works with $X^\top X$, with the conditioning of the normal equations.

Lecture 4. Least Squares: QR Decomposition

4.1 QR decomposition and least squares

Cholesky on the normal equations is one way to do least squares; the QR decomposition is another, and is what R's `lm` uses. Let $X \in \mathbb{R}^{n\times p}$ have full column rank ($n \geq p$).

Definition (QR decomposition)

- **Full QR**: $X = QR$ with $Q \in \mathbb{R}^{n\times n}$ orthogonal and $R \in \mathbb{R}^{n\times p}$ upper triangular (its lower $(n-p)\times p$ block is 0). The first p columns of Q are an orthonormal basis of the column space $\mathcal{R}(X)$, the last $n-p$ of $\mathcal{N}(X^\top) = \mathcal{R}(X)^\perp$.
- **Reduced QR**: $X = Q_1 R_1$ with $Q_1 \in \mathbb{R}^{n\times p}$, $Q_1^\top Q_1 = I_p$ (but $Q_1 Q_1^\top \neq I_n$), and $R_1 \in \mathbb{R}^{p\times p}$ upper triangular with positive diagonal.

Since $X^\top X = R^\top Q^\top Q R = R_1^\top R_1$, a QR decomposition of X gives the Cholesky factor of the Gram matrix for free.

Least squares via QR. The normal equations $X^\top X\beta = X^\top y$ become $R_1^\top R_1 \beta = R_1^\top Q_1^\top y$, and since R_1 is invertible,

$$R_1\beta = Q_1^\top y,$$

a single back substitution. (Equivalently, $\|y - X\beta\|^2 = \|Q^\top y - R\beta\|^2 = \|Q_1^\top y - R_1\beta\|^2 + \|Q_2^\top y\|^2$, minimized when the first term vanishes, and the residual sum of squares is $\|Q_2^\top y\|^2$.) This is more accurate than solving the normal equations because

$$\kappa(X^\top X) = \kappa(X)^2,$$

since the singular values of $X^\top X$ are the squares of those of X. Standard errors need $(X^\top X)^{-1} = R_1^{-1} R_1^{-\top}$, again via triangular solves.

4.2 Gram–Schmidt

Every full-column-rank X has a reduced QR decomposition: it is the Gram–Schmidt orthonormalization of the columns $x_1, \ldots, x_p$.

(1) $q_1 = x_1/\|x_1\|_2$.

(2) For $k = 2, \ldots, p$: $v_k = x_k - \sum_{j<k} \langle q_j, x_k\rangle q_j$ and $q_k = v_k/\|v_k\|_2$.

The q_k are orthonormal with $\operatorname{span}\{x_1, \ldots, x_k\} = \operatorname{span}\{q_1, \ldots, q_k\}$ for each k. With $r_{jk} = \langle q_j, x_k\rangle$ $(j < k)$, $r_{kk} = \|v_k\|_2$ and $r_{jk} = 0$ $(j > k)$, step (2) reads $x_k = \sum_{j\leq k} r_{jk} q_j$, i.e. $X = Q_1 R_1$.

Classical vs modified. Classical Gram–Schmidt (CGS) computes all $\langle q_j, x_k\rangle$ with the original x_k. When columns are nearly collinear, rounding errors make the computed q's far from orthogonal. **Modified** Gram–Schmidt (MGS) subtracts the projections one at a time, projecting the *current* partially orthogonalized vector:

$$v_k^{(1)} = x_k, \qquad v_k^{(j+1)} = v_k^{(j)} - \langle q_j, v_k^{(j)}\rangle q_j, \quad j = 1, \ldots, k-1, \qquad q_k = v_k^{(k)}/\|v_k^{(k)}\|.$$

In exact arithmetic the two agree; in floating point MGS loses much less orthogonality, though it is not fully immune. Both cost $\sum_k 4n(k-1) \approx 2np^2$ flops and give only the reduced QR.

Example 4.1 (Where CGS breaks)

Let $x_1 = (1, \epsilon, 0, 0)^\top$, $x_2 = (1, 0, \epsilon, 0)^\top$, $x_3 = (1, 0, 0, \epsilon)^\top$ with ϵ^2 below the unit roundoff, so that $[1 + \epsilon^2] = 1$. Then $r_{11} = [\sqrt{1 + \epsilon^2}] = 1$ and $q_1 = x_1$; the computed r_{11} has dropped the ϵ^2 that should have entered later inner products.

- $k = 2$: $r_{12} = q_1^\top x_2 = 1$, $v_2 = (0, -\epsilon, \epsilon, 0)^\top$, $q_2 = (0, -1, 1, 0)^\top/\sqrt{2}$.
- $k = 3$, CGS: $r_{13} = q_1^\top x_3 = 1$ and $r_{23} = q_2^\top x_3 = 0$ (both with the *original* x_3), so $v_3 = (0, -\epsilon, 0, \epsilon)^\top$ and $q_3 = (0, -1, 0, 1)^\top/\sqrt{2}$. Now $q_2^\top q_3 = \frac{1}{2}$: orthogonality is completely lost.
- $k = 3$, MGS: after removing q_1, $v_3^{(2)} = (0, -\epsilon, 0, \epsilon)^\top$ and $q_2^\top v_3^{(2)} = \epsilon/\sqrt{2} \neq 0$, so the second projection is removed: $v_3 = (0, -\frac{\epsilon}{2}, -\frac{\epsilon}{2}, \epsilon)^\top$, $q_3 = (0, -1, -1, 2)^\top/\sqrt{6} \perp q_2$.

The true $q_2^\top x_3$ is a tiny nonzero number that CGS rounds away; MGS recomputes it from the partially orthogonalized vector, where it is no longer negligible relative to the vector's size. MGS fails in turn when two columns are almost parallel, e.g. $x_1 = (0.7, 0.7 + \epsilon_{\text{mach}})^\top$ and $x_2 = (1/\sqrt{2}, 1/\sqrt{2})^\top$: then $v_2 = x_2 - \langle q_1, x_2\rangle q_1$ is a difference of nearly equal vectors whose computed direction is rounding noise. With only two columns CGS and MGS perform the same operations, so CGS fails too. Householder QR, which never subtracts projections, gives an orthogonal Q to machine precision in both cases.

4.3 Householder reflections

Motivation. Solving least squares through $X^\top X$ squares the condition number. Orthogonal transformations preserve lengths, so reducing X to triangular form by reflections (QR) solves least squares accurately.

Proposition 4.1 (Householder reflector)

For $x, w \in \mathbb{R}^n$ with $\|x\|_2 = \|w\|_2$ and $x \neq w$, let $u = (x - w)/\|x - w\|_2$ and $H = I - 2uu^\top$. Then H is symmetric and orthogonal and $Hx = w$.

Proof. $H^\top = H$ and $H^2 = I - 4uu^\top + 4u(u^\top u)u^\top = I$. Next, $\|x - w\|^2 = 2x^\top x - 2w^\top x$ (using $x^\top x = w^\top w$), so

$$Hx = x - 2\frac{(x - w)^\top x}{\|x - w\|^2}(x - w) = x - \frac{x^\top x - w^\top x}{x^\top x - w^\top x}(x - w) = w.$$

■

Geometrically H reflects across the hyperplane orthogonal to u. Householder QR chooses H_1 with $w = \pm\|x_1\|e_1$ to zero the first column below the diagonal, then $H_2 = \text{diag}(1, \tilde{H}_2)$ acting on rows $2{:}n$ to zero the second column, and so on:

$$H_p \cdots H_2 H_1 X = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}, \qquad Q = H_1 H_2 \cdots H_p.$$

This gives the *full* QR. The sign of w is chosen opposite to the first entry of x, to avoid cancellation in $x - w$: with $w = -\text{sign}(x_1)\|x\|e_1$, the first entry of $x - w$ is $x_1 + \text{sign}(x_1)\|x\|$, a sum of two numbers of the same sign. With the other choice it would be $x_1 - \text{sign}(x_1)\|x\|$, which is catastrophically small when x is nearly parallel to e_1, and u would be computed from noise.

Implementation and cost. The matrices H_j are never formed: $HX = X - 2u(u^\top X)$ costs $4np$ flops. At stage j, computing $\tilde{u}_j$ costs $3(n-j+1)$ and applying $\tilde{H}_j$ to the $(n-j+1)\times(p-j+1)$ block costs $4(n-j+1)(p-j+1)$; in total

$$\sum_{j=1}^{p} 4(n-j+1)(p-j+1) \approx 2np^2 - \tfrac{2}{3}p^3 \text{ flops.}$$

The algorithm works in place: R_1 overwrites the upper triangle, and the Householder vectors (normalized so their first entry is 1) are stored below the diagonal. To compute $Q^\top y$ or Qv one applies the reflections successively; forming Q explicitly costs another $2np^2 - \frac{2}{3}p^3$. Householder QR is backward stable and is what LAPACK (`geqrf`, or `geqp3` with column pivoting) and R's `qr` use.

Least squares without forming Q. Apply each reflection to y as soon as it is computed (cost $4(n-j+1)$ at stage j). At the end y has been overwritten by $Q^\top y = \binom{Q_1^\top y}{Q_2^\top y}$, in place and with no $n \times n$ matrix ever stored. Then

- $\hat{\beta}$ solves $R_1\beta = Q_1^\top y$ (the first p entries), by back substitution;
- $\mathrm{RSS} = \|Q_2^\top y\|^2$ is the sum of squares of the last $n-p$ entries, so $\hat{\sigma}^2 = \mathrm{RSS}/(n-p)$, or RSS/n for the maximum likelihood estimator;
- $\widehat{\mathrm{Cov}}(\hat{\beta}) = \hat{\sigma}^2 R_1^{-1} R_1^{-\top}$, from triangular solves with the R_1 already in the upper triangle.

The whole fit costs $2np^2 - \frac{2}{3}p^3 + O(np)$ flops and $O(np)$ memory.

4.4 Givens rotations

A Givens rotation $G(i,k,\theta)$ is the identity except for the entries $c, s, -s, c$ in rows and columns i, k, with $c = \cos\theta$, $s = \sin\theta$; it is orthogonal. For $y = G^\top x$,

$$y_i = cx_i - sx_k, \quad y_k = sx_i + cx_k, \quad y_j = x_j \ (j \neq i,k),$$

and the choice $c = x_i/\sqrt{x_i^2 + x_k^2}$, $s = -x_k/\sqrt{x_i^2 + x_k^2}$ makes $y_k = 0$. Applying $G^\top$ to a matrix changes only two rows ($6m$ flops). QR by Givens zeros one entry at a time, $G_t^\top \cdots G_1^\top X = \binom{R_1}{0}$, at a cost of about $3np^2 - p^3$ flops: more than Householder, but well suited to sparse or structured matrices (e.g. updating a QR when a row is added), since it touches only the entries it needs. Each rotation can be stored as a single number in the zeroed position.

4.5 Conditioning of least squares

For a nonsquare X, $\kappa(X) = \sigma_{\max}(X)/\sigma_{\min}(X) = \sigma_{\max}(X)\sigma_{\max}(X^\dagger)$, with $X^\dagger$ the Moore–Penrose pseudoinverse. The condition number of the least squares *problem* depends on the residual; it is

$$\kappa(X) + \frac{\kappa(X)^2 \tan\theta}{\eta}, \qquad \cos\theta = \frac{\|X\beta\|}{\|y\|}, \quad \eta = \frac{\|X\|\|\beta\|}{\|X\beta\|}.$$

When X is nearly collinear and the residuals are not small, the $\kappa(X)^2$ term dominates, so stable algorithms matter.

Example 4.2

$X = \begin{pmatrix} 1 & 1 \\ 10^{-3} & 0 \\ 0 & 10^{-3} \end{pmatrix}$ has full rank, but $X^\top X = \begin{pmatrix} 1+10^{-6} & 1 \\ 1 & 1+10^{-6} \end{pmatrix}$ rounds to the singular $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ with six significant digits. Forming the normal equations destroys the information, while QR applied to X itself does not.

method	flops	stability	notes
Cholesky on $X^\top X$	$np^2 + \frac{1}{3}p^3$	least stable	fastest; $X^\top X$ can be updated with streaming data
Householder QR	$2np^2 - \frac{2}{3}p^3$	stable	default in R, MATLAB, Julia
MGS	$2np^2$	stable-ish	gives Q_1 explicitly
SVD	$4n^2p + 8np^2 + 9p^3$	most stable	also handles rank deficiency

There is no universal gold standard: Cholesky is twice as fast for $n \gg p$ and needs only $X^\top X$ (which fits in memory even when X does not), while QR is more accurate.

4.6 Underdetermined least squares

If $\operatorname{rank} X < p$ (in particular $n < p$), the normal equations have infinitely many solutions, and an extra criterion is needed.

- **Minimum ℓ_2 norm**: minimize $\|\beta\|_2$ subject to $X^\top X\beta = X^\top y$. The solution is $\hat\beta = X^\dagger y$, which for full row rank X ($n < p$) is $X^\top(XX^\top)^{-1}y$. It is also the limit of ridge regression: $\hat\beta = \lim_{\lambda\downarrow 0}(X^\top X + \lambda I)^{-1}X^\top y$.
- **Minimum ℓ_0 "norm"**: minimize $\|\beta\|_0 = \#\{j : \beta_j \neq 0\}$, the sparsest solution. This is best subset selection, a combinatorial problem.
- **Minimum ℓ_1 norm**: a convex relaxation (basis pursuit, related to the lasso), solvable by linear programming, which under suitable conditions recovers the sparsest solution.

Lecture 5. Iterative Methods for Linear Systems

5.1 Direct versus iterative methods

Direct methods (LU, Cholesky, QR, SVD) have a cost fixed in advance and are accurate; they suit dense matrices of small or moderate size. **Iterative methods** (Jacobi, Gauss–Seidel, SOR, conjugate gradient, GMRES) produce a sequence of approximations whose accuracy depends on the number of iterations; they suit very large, sparse or structured systems, parallel computing, and warm starts.

Example 5.1 (PageRank)

Let $a_{ij} = 1$ if web page i links to page j, and $r_i = \sum_j a_{ij}$ the out-degree. A random surfer at a page with $r_i > 0$ follows a random link with probability p and jumps to a uniformly random page otherwise; from a page with no links, the surfer jumps uniformly. This Markov chain has transition probabilities

$$p_{ij} = \frac{1-p}{n} + p\frac{a_{ij}}{r_i} \qquad (a_{ij}/r_i := 1/n \text{ if } r_i = 0),$$

and the page scores are its stationary distribution: $x^\top P = x^\top$, i.e. $(I - P^\top)x = 0$. With $n \approx 10^9$ pages, LU would take $\frac{2}{3}(10^9)^3/10^{12}$ seconds, about 20 million years on a teraflop machine. Iterative methods, which only need products with the sparse link matrix, make it feasible. The dense matrix $I - P^\top$ is never formed: row i of $(I - P^\top)x$ is $x_i - \frac{1-p}{n}\sum_{j:r_j>0} x_j - \frac{1}{n}\sum_{j:r_j=0} x_j - p\sum_{j\to i} x_j/r_j$, where the last sum runs over the pages linking to i (a column of the sparse A). The two global sums are kept as running totals, so a full Gauss–Seidel or SOR sweep costs $O(n + \mathrm{nnz}(A))$.

5.2 Splitting and convergence

Split $A = M - K$ with M invertible and easy to invert. Then $Ax = b$ iff $x = M^{-1}Kx + M^{-1}b$, so a solution is a fixed point of

$$x^{(t+1)} = Rx^{(t)} + c, \qquad R = M^{-1}K, \quad c = M^{-1}b.$$

Norms. A vector norm satisfies $\|x\| \geq 0$, $\|x\| = 0 \iff x = 0$, $\|cx\| = |c|\|x\|$ and the triangle inequality; common choices are ℓ_1, ℓ_2, ℓ_∞. A matrix norm is also required to be submultiplicative, $\|AB\| \leq \|A\|\|B\|$. Examples: the Frobenius norm $\|A\|_F = \sqrt{\mathrm{tr}(AA^\top)}$, and the **operator norm** $\|A\| = \sup_{y\neq 0}\|Ay\|/\|y\|$ induced by a vector norm, with

$$\|A\|_1 = \max_j \sum_i |a_{ij}|, \qquad \|A\|_\infty = \max_i \sum_j |a_{ij}|, \qquad \|A\|_2 = \sqrt{\rho(A^\top A)},$$

where $\rho(B) = \max_i|\lambda_i(B)|$ is the **spectral radius**.

If $x^\star$ solves $Ax = b$, then $x^{(t+1)} - x^\star = R(x^{(t)} - x^\star)$, so $\|x^{(t+1)} - x^\star\| \leq \|R\|\|x^{(t)} - x^\star\|$, and $\|R\| < 1$ for some operator norm implies convergence.

Theorem 5.1

For every operator norm, $\rho(A) \le \|A\|$. Conversely, for every $\epsilon > 0$ there is an operator norm with $\|A\| \le \rho(A) + \epsilon$. Hence $\rho(A) = \inf\|A\|$ over operator norms, and $\rho(A) < 1$ iff $\|A\| < 1$ for some operator norm.

Proof. If $Av = \lambda v$ with $v \neq 0$ (complex in general), then $|\lambda|\|v\| = \|Av\| \le \|A\|\|v\|$.

For the converse, use the Schur decomposition $A = STS^{-1}$ with T upper triangular whose diagonal holds the eigenvalues. For $\delta > 0$ let $D_\delta = \mathrm{diag}(1, \delta, \dots, \delta^{n-1})$. Then $T_\delta = D_\delta^{-1} T D_\delta = (SD_\delta)^{-1} A (SD_\delta)$ has entries $\delta^{j-i} t_{ij}$ $(j \ge i)$: the off-diagonal part shrinks as $\delta \to 0$. Define $\|x\|_\delta = \|(SD_\delta)^{-1} x\|_\infty$; it is a vector norm because $M = (SD_\delta)^{-1}$ is invertible (nonnegativity, homogeneity and the triangle inequality transfer from $\|\cdot\|_\infty$, and $\|Mx\|_\infty = 0 \iff Mx = 0 \iff x = 0$). Substituting $y = (SD_\delta)^{-1} x$, its induced norm is

$$\|A\|_\delta = \sup_{y \neq 0} \frac{\|(SD_\delta)^{-1} A (SD_\delta) y\|_\infty}{\|y\|_\infty} = \|T_\delta\|_\infty = \max_i \sum_{j \ge i} |t_{ij}| \delta^{j-i},$$

which is at most $\max_i |t_{ii}| + \epsilon = \rho(A) + \epsilon$ for small δ. ∎

Corollary 5.2

If A is invertible, the iteration $x^{(t+1)} = Rx^{(t)} + c$ converges to $A^{-1}b$ for every $x^{(0)}$ if and only if $\rho(R) < 1$.

Proof. If $\rho(R) < 1$, pick an operator norm with $\|R\| < 1$ by Theorem 5.1; then $\|x^{(t)} - x^\star\| \le \|R\|^t \|x^{(0)} - x^\star\| \to 0$. (This direction does not need A invertible; the limit is then some solution.) Conversely, if $|\lambda| \ge 1$ for an eigenpair $Rv = \lambda v$, start at $x^{(0)} = x^\star + v$: then $\|x^{(t)} - x^\star\| = |\lambda|^t \|v\| \ge \|v\| \neq 0$, so the iteration does not converge. ∎

5.3 Classical iterations

Write $A = L + D + U$ (strictly lower, diagonal, strictly upper parts).

Jacobi. $M = D$, $K = -(L + U)$:

$$x_i^{(t+1)} = \frac{1}{a_{ii}} \Big(b_i - \sum_{j \neq i} a_{ij} x_j^{(t)} \Big), \qquad x^{(t+1)} = x^{(t)} + D^{-1}(b - Ax^{(t)}).$$

Gauss–Seidel. $M = D + L$ (lower triangular, easy to invert by forward substitution), $K = -U$: newly computed components are used immediately,

$$x_i^{(t+1)} = \frac{1}{a_{ii}} \Big(b_i - \sum_{j<i} a_{ij} x_j^{(t+1)} - \sum_{j>i} a_{ij} x_j^{(t)} \Big).$$

It is the linear-system analogue of coordinate descent. In code the two methods differ by one detail: a loop over i that writes the new x_i back into x before computing the next component is Gauss–Seidel, while one that stores all new components separately and replaces x only after the sweep is Jacobi. Gauss–Seidel needs no second vector, but its sweep is inherently sequential, whereas Jacobi vectorizes and parallelizes.

Successive over-relaxation (SOR). A weighted combination of the Gauss–Seidel update and the old value:

$$x_i^{(t+1)} = (1-\omega)x_i^{(t)} + \frac{\omega}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j^{(t+1)} - \sum_{j>i} a_{ij}x_j^{(t)}\Big),$$

i.e. $M = \frac{1}{\omega}D + L$, $K = \frac{1-\omega}{\omega}D - U$. $\omega = 1$ is Gauss–Seidel; $\omega > 1$ (over-relaxation) can accelerate convergence substantially.

Proposition 5.3

If A is strictly diagonally dominant by rows, $|a_{ii}| > \sum_{j\neq i}|a_{ij}|$, then the Jacobi and Gauss–Seidel iterations converge for every starting point.

Proof. *Jacobi.* $R = -D^{-1}(L+U)$ has $\|R\|_\infty = \max_i \sum_{j\neq i}|a_{ij}|/|a_{ii}| < 1$. *Gauss–Seidel.* Let $Rv = \lambda v$ with $R = -(D+L)^{-1}U$, so $-Uv = \lambda(D+L)v$, and choose i with $|v_i| = \|v\|_\infty = 1$. The i-th row gives $\lambda a_{ii}v_i = -\sum_{j>i} a_{ij}v_j - \lambda\sum_{j<i} a_{ij}v_j$, hence

$$|\lambda|\Big(|a_{ii}| - \sum_{j<i}|a_{ij}|\Big) \le \sum_{j>i}|a_{ij}|,$$

and the right side is smaller than the bracket by diagonal dominance. So $|\lambda| < 1$ and $\rho(R) < 1$. ■

Jacobi also converges if A is strictly diagonally dominant by *columns*, $|a_{jj}| > \sum_{i\neq j}|a_{ij}|$: then $A^\top$ is row dominant, and $R = -D^{-1}(L+U)$ is similar to $DRD^{-1} = -(L+U)D^{-1}$, whose $\|\cdot\|_1$ (maximum column sum) is $\max_j \sum_{i\neq j}|a_{ij}|/|a_{jj}| < 1$. Similar matrices have the same spectral radius.

Example 5.2 (Jacobi for the second-difference matrix)

Let $A = \mathrm{tridiag}(-1, 2, -1) \in \mathbb{R}^{n\times n}$, whose eigenvalues are $\lambda_i = 2 - 2\cos\frac{i\pi}{n+1}$ with orthonormal eigenvectors v_i. Jacobi has $D = 2I$ and $R = -D^{-1}(L+U) = I - \frac{1}{2}A$, so R has the same eigenvectors and eigenvalues

$$\mu_i = 1 - \tfrac{1}{2}\lambda_i = \cos\frac{i\pi}{n+1}, \qquad \rho(R) = \cos\frac{\pi}{n+1} < 1.$$

The method converges, but slowly: $\rho(R) \approx 1 - \frac{\pi^2}{2(n+1)^2}$, so reducing the error by a constant factor takes $O(n^2)$ sweeps. Writing the error as $\varepsilon^{(k)} = \sum_i a_i^{(k)} v_i$, the recursion $\varepsilon^{(k+1)} = R\varepsilon^{(k)}$ gives $a_i^{(k)} = \mu_i^k a_i^{(0)}$: each mode decays geometrically at its own rate $|\mu_i|$. Modes with i near $(n+1)/2$ ($\mu_i \approx 0$) die almost at once, while the smoothest mode $i = 1$ and the most oscillatory mode $i = n$ ($\mu_n = -\mu_1$, sign-alternating) persist. This observation is the starting point of multigrid methods, which treat smooth error components on coarser grids.

Cost. One sweep costs $2n^2$ flops for a dense A, so iteration beats LU only if it converges in $o(n)$ sweeps. The real gain is for sparse or structured A, where Av costs $O(n)$ or $O(n\log n)$ and A need not even be stored (e.g. a finite-difference operator $(Av)_i = v_i - v_{i+1}$).

5.4 Conjugate gradient

For symmetric positive definite A, solving $Ax = b$ is equivalent to minimizing the convex quadratic $\frac{1}{2}x^\top Ax - b^\top x$, whose gradient is $Ax - b$. The **conjugate gradient** method minimizes it along mutually A-conjugate directions and, in exact arithmetic, terminates in at most n steps; in practice, especially with preconditioning, it converges in far fewer. It is the method of choice for huge

structured positive definite systems. (In a classical plasma physics example, Gauss–Seidel needed 208,000 iterations, block SOR 765, and incomplete-Cholesky-preconditioned CG 25.) The details belong to an advanced course.

A numerical test on the Poisson matrix (a sparse, symmetric positive definite block tridiagonal matrix from the discretized Laplacian) shows the pattern: Jacobi needs thousands of iterations, Gauss–Seidel and SOR fewer, and CG far fewer, all much cheaper than a dense Cholesky solve. Once again: *exploit structure*.

Lecture 6. Nonlinear Equations

We want x with $f(x) = 0$ for nonlinear f. There is no direct method; we iterate. Root finding and optimization are closely linked: a critical point of a differentiable g is a root of $f = g'$ (or ∇g). Statistical uses include maximum likelihood, nonlinear least squares, Bayes and MAP estimation. Examples: a root of $f(x) = x^4 - \sin x$; the maximizer of $g(x) = \log x/(1+x)$, a root of $g'(x) = \{1 + x^{-1} - \log x\}/(1+x)^2$.

6.1 Bisection

If f is continuous on $[l, r]$ with $f(l)f(r) \le 0$, the intermediate value theorem gives a root in $[l, r]$. Evaluate f at the midpoint $m = (l+r)/2$: stop if $f(m) = 0$; otherwise replace r by m if $f(l)f(m) < 0$ and l by m otherwise. After n steps the bracketing interval has length $2^{-n}(r^{(0)} - l^{(0)})$, so the midpoint is within that distance of a root. Bisection is slow (one bit per step) but always works under these weak assumptions. A typical use is computing a quantile: solve $F(x) - u = 0$ for a continuous cdf F. If F itself is only available as $\int_{-\infty}^{x} f$, each evaluation is a numerical integral (Lecture 8); this is how the inverse cdf method of Lecture 9 can be used when F^{-1} has no closed form.

6.2 Fixed-point iteration

x is a **fixed point** of F if $F(x) = x$. To find a root of f, choose F with $f(x) = 0 \iff F(x) = x$ and iterate $x^{(k+1)} = F(x^{(k)})$. (The iterative methods of Lecture 5 are the case $F(x) = Rx + c$.)

Theorem 6.1 (Contraction mapping)

Let $I \subset \mathbb{R}$ be a closed interval and $F : I \to I$ with $|F(x) - F(y)| \le L|x - y|$ for all $x, y \in I$, where $0 \le L < 1$. Then F has a unique fixed point $x^\star \in I$, the iteration converges to it from every $x^{(0)} \in I$, and

$$|x^{(n)} - x^\star| \le \frac{L^n}{1-L}|x^{(1)} - x^{(0)}|.$$

Proof. $|x^{(k+1)} - x^{(k)}| = |F(x^{(k)}) - F(x^{(k-1)})| \le L|x^{(k)} - x^{(k-1)}| \le \cdots \le L^k|x^{(1)} - x^{(0)}|$. For $m > n$, by the triangle inequality,

$$|x^{(m)} - x^{(n)}| \le \sum_{k=n}^{m-1} L^k|x^{(1)} - x^{(0)}| \le \frac{L^n}{1-L}|x^{(1)} - x^{(0)}|. \qquad (*)$$

So the sequence is Cauchy and converges to some $x^\star$, which lies in I because I is closed. F is continuous, so $x^\star = \lim x^{(k+1)} = \lim F(x^{(k)}) = F(x^\star)$. If $y^\star$ were another fixed point, $|x^\star - y^\star| = |F(x^\star) - F(y^\star)| \le L|x^\star - y^\star|$ forces $x^\star = y^\star$. Letting $m \to \infty$ in $(*)$ gives the bound. ■

The geometric rate L^n is called **linear convergence**.

Proposition 6.2

If F is differentiable on $I = [a, b]$, then $|F(x) - F(y)| \le L|x - y|$ for all $x, y \in I$ if and only if $|F'(x)| \le L$ for all $x \in I$. Hence the best Lipschitz constant is $\sup_I |F'|$.

Proof. ($\Rightarrow$) For $x \in I$ and $y_n \to x$ in I, $y_n \ne x$, the difference quotients are bounded by L in absolute value, and so is their limit $F'(x)$.

($\Leftarrow$) For $x \ne y$, the mean value theorem gives c between them with $|F(x) - F(y)| = |F'(c)||x - y| \le L|x - y|$. ■

Example 6.1 (Relaxed fixed-point iteration)

To solve $f(x) = 0$, iterate $x^{(t+1)} = x^{(t)} + \alpha f(x^{(t)})$, $\alpha \neq 0$, i.e. $F(x) = x + \alpha f(x)$. By Proposition 6.2 and Theorem 6.1, it converges whenever F maps an interval I around the root into itself and $\sup_I |1 + \alpha f'(x)| < 1$. This needs f' of constant sign on I; if, say, $0 < m \leq f' \leq M$ there, every $\alpha \in (-2/M, 0)$ works, and $\alpha = -2/(m + M)$ gives the best constant $L = (M - m)/(M + m)$. So the free parameter α turns f into a contraction that the plain iteration $x + f(x)$ need not be, and tunes the rate. Newton's method is the adaptive choice $\alpha = -1/f'(x^{(t)})$, which makes $F'(x^\star) = 0$. If $f = g'$ for a twice differentiable g to be minimized, the iteration with $\alpha = -\gamma$ is exactly gradient descent $x^{(t+1)} = x^{(t)} - \gamma g'(x^{(t)})$ (Lecture 7), and the condition above is the familiar step-size bound $\gamma < 2/\sup g''$.

Example 6.2

For $f(x) = x^4 - \sin x$ on $(0, 1]$, both $F_1(x) = (\sin x)^{1/4}$ and $F_2(x) = \sin x / x^3$ have the root as a fixed point. On $I = [0.2, 1]$, F_1 maps I into itself and $F_1'(x) = \cos x / \{4(\sin x)^{3/4}\}$ is decreasing with $0 < F_1'(1) < F_1'(0.2) < 1$, so F_1 is a contraction and the iteration converges. F_2 maps $[0.9, 1]$ onto the larger $[0.841, 1.075]$ with $F_2'(1) < -1$: it expands, and the iteration diverges. The choice of F matters.

6.3 Newton's method

Motivation. Fixed-point iteration converges only linearly. Newton's method replaces f by its tangent line at each step, which doubles the number of correct digits near a simple root; the price is computing derivatives and the risk of divergence from a bad start.

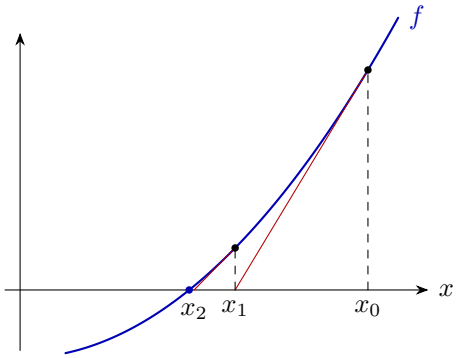


Figure. *Newton's method replaces f by its tangent line at x_k and takes the tangent's zero as $x_{k+1} = x_k - f(x_k)/f'(x_k)$. Near a simple root the error is roughly squared at each step.*

Linearizing f at the current iterate, $0 = f(x^\star) \approx f(x^{(k)}) + f'(x^{(k)})(x^\star - x^{(k)})$, suggests

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}.$$

This is fixed-point iteration with $F(x) = x - f(x)/f'(x)$, for which

$$F'(x) = \frac{f(x) f''(x)}{f'(x)^2}.$$

Theorem 6.3 (Local quadratic convergence)

Let $f(x^\star) = 0$, $f'(x^\star) \neq 0$, and f be three times continuously differentiable near $x^\star$. Then for $x^{(0)}$ close enough to $x^\star$, Newton's iterates converge to $x^\star$ and

$$\lim_{k\to\infty} \frac{|x^{(k+1)} - x^\star|}{|x^{(k)} - x^\star|^2} = \frac{1}{2}|F''(x^\star)| = \frac{|f''(x^\star)|}{2|f'(x^\star)|}.$$

Proof. $F'(x^\star) = 0$ since $f(x^\star) = 0$; by continuity $|F'| \le \frac{1}{2}$ on a closed neighbourhood I of $x^\star$, which F then maps into itself, so Theorem 6.1 gives convergence. By Taylor's theorem,

$$x^{(k+1)} - x^\star = F(x^{(k)}) - F(x^\star) = F'(x^\star)(x^{(k)} - x^\star) + \tfrac{1}{2}F''(\tilde{x})(x^{(k)} - x^\star)^2 = \tfrac{1}{2}F''(\tilde{x})(x^{(k)} - x^\star)^2,$$

with $\tilde{x}$ between $x^{(k)}$ and $x^\star$. Divide and let $k \to \infty$; $F''(x^\star) = f''(x^\star)/f'(x^\star)$ follows by differentiating F'. ∎

Quadratic convergence roughly doubles the number of correct digits per step. But it is only *local*: from a poor starting point Newton's method can wander off, converge to an unintended root, or diverge (e.g. seeking a root of $\cos x$ from different starting points). Safeguards (combining with bisection, line search) are used in practice.

6.4 Systems of equations

For $f : \mathbb{R}^n \to \mathbb{R}^n$ with Jacobian $Jf(x) = (\partial f_i/\partial x_j)$, Newton's method is

$$x^{(k+1)} = x^{(k)} - [Jf(x^{(k)})]^{-1} f(x^{(k)}),$$

implemented by solving the linear system $Jf(x^{(k)})\Delta x = -f(x^{(k)})$ (e.g. by LU) at each step, never by inverting. Cheaper variants replace the Jacobian:

- modified Newton: the fixed $J = Jf(x^{(0)})$ (factor once);
- Jacobi: $\operatorname{diag} Jf(x^{(k)})$;
- Gauss–Seidel: the lower triangle of $Jf(x^{(k)})$;
- SOR: $\frac{1}{\omega}\operatorname{diag} Jf(x^{(k)})$ plus the strict lower triangle.

These converge more slowly (typically linearly) but each step is much cheaper.

Lecture 7. Optimization

We consider $\min_{x\in\mathbb{R}^d} f(x)$; maximization is minimization of $-f$.

7.1 Golden section search

For $d = 1$, let f be continuous and **strictly unimodal** on $[a, b]$: $f(\lambda x+(1-\lambda)y) < \max\{f(x), f(y)\}$ for $x \neq y$, $\lambda \in (0, 1)$. Keep three points $a < x_1 < b$ with $f(x_1) < \min\{f(a), f(b)\}$, so the minimum lies in (a, b). Place a new point x_0 in the longer of (a, x_1) and (x_1, b); say $a < x_0 < x_1$.

- If $f(x_0) > f(x_1)$, the minimum is in (x_0, b): set $(a, x_1, b) \leftarrow (x_0, x_1, b)$.
- Otherwise it is in (a, x_1): set $(a, x_1, b) \leftarrow (a, x_0, x_1)$.

To make both outcomes equally good and keep the configuration self-similar, require $b-x_0 = x_1-a$ and that the interior point divides the new interval in the same proportion. With $\alpha = (x_0-a)/(b-a)$ and $y = (x_1 - x_0)/(b - a)$, this gives $\alpha + y + \alpha = 1$ and $1 : \alpha = (y + \alpha) : y$, whose solution is

$$\alpha = \frac{3-\sqrt{5}}{2}, \qquad 1-\alpha = \frac{\sqrt{5}-1}{2} \approx 0.618,$$

the golden section. Each step shrinks the interval by the factor 0.618 with one function evaluation: slow (linear) but guaranteed to find the global minimum of a unimodal function.

7.2 Newton's method

At a minimizer, $\nabla f(x^\star) = 0$. Applying Newton–Raphson to ∇f is equivalent to minimizing the local quadratic model

$$f(x) \approx f(x^{(t)}) + \nabla f(x^{(t)})^\top (x - x^{(t)}) + \tfrac{1}{2}(x - x^{(t)})^\top \nabla^2 f(x^{(t)})(x - x^{(t)}),$$

giving the **naive Newton** iteration

$$x^{(t+1)} = x^{(t)} - [\nabla^2 f(x^{(t)})]^{-1} \nabla f(x^{(t)}).$$

It converges quadratically near a minimizer with positive definite Hessian. But it is not a descent method: it is equally happy to go uphill and can converge to a maximum or a saddle, or diverge, depending on the start (try $f = \sin$).

7.3 Practical Newton: positive definite approximations and line search

Motivation. Pure Newton steps for optimization can move uphill when the Hessian is not positive definite, or overshoot far from the optimum. Modifying the Hessian and choosing the step length by line search keep the fast local convergence while guaranteeing descent.

Replace the Hessian by a positive definite $H^{(t)}$ and choose a step size s:

$$x^{(t+1)} = x^{(t)} + s\,\Delta x^{(t)}, \qquad \Delta x^{(t)} = -[H^{(t)}]^{-1} \nabla f(x^{(t)}).$$

Proposition 7.1 (Descent direction)

If $H^{(t)}$ is positive definite and $\nabla f(x^{(t)}) \neq 0$, then $f(x^{(t)} + s\Delta x^{(t)}) < f(x^{(t)})$ for all sufficiently small $s > 0$.

Proof. By Taylor's theorem, $f(x^{(t)}+s\Delta x^{(t)})-f(x^{(t)}) = -s\,\nabla f^\top [H^{(t)}]^{-1}\nabla f+o(s)$, and the quadratic form is positive. ■

($\{\nabla f^\top H^{-1}\nabla f\}^{1/2}$ is the **Newton decrement**.) For strictly convex f the true Hessian already works (**damped Newton**), but a line search is still needed for global convergence. Taking $H^{(t)} = I$ gives gradient descent.

Line search. Given the direction, choose s: exactly (minimize $f(x+s\Delta x)$, e.g. by golden section), or approximately by step halving or **backtracking** (Armijo): with $\alpha \in (0, \frac{1}{2})$ and $\beta \in (0, 1)$, start from $s = 1$ and set $s \leftarrow \beta s$ while

$$f(x + s\Delta x) > f(x) + \alpha s \nabla f(x)^\top \Delta x.$$

The condition accepts s once f lies below a line whose slope is a fraction α of the initial slope, which always happens for small s by Proposition 7.1. The direction is computed once per iteration; the line search costs only function evaluations. A common implementation slip: if the trial point $x_s = x + s\Delta x$ is formed first, the right side of the test is $f(x) + \alpha\nabla f(x)^\top (x_s - x)$, which already contains s; writing $\alpha s \nabla f(x)^\top (x_s - x)$ counts s twice and makes the test too lenient. (For IRLS, the trial point for step s can be computed by refitting the weighted least squares with working response $z = \eta + sW^{-1}(y - \mu)$, since the fit is linear in z and returns exactly $\beta + s\Delta\beta$.)

7.4 Fisher scoring

For maximum likelihood, $f = -\ell(\theta)$. **Fisher scoring** replaces $-\nabla^2\ell$ by the expected information

$$I(\theta) = E[-\nabla^2\ell(\theta)] = E[\nabla\ell(\theta)\nabla\ell(\theta)^\top] \succeq 0,$$

(the equality holds when differentiation and expectation can be interchanged), giving

$$\theta^{(t+1)} = \theta^{(t)} + s[I(\theta^{(t)})]^{-1}\nabla\ell(\theta^{(t)}).$$

Example 7.1 (Logistic regression)

$y_i \sim \text{Ber}(p_i)$ with $\text{logit}\, p_i = \eta_i = x_i^\top \beta$. Then

$$\ell(\beta) = \sum_i \{y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta})\}, \qquad \nabla\ell = X^\top (y - p),$$

$$-\nabla^2\ell = X^\top W X, \qquad W = \text{diag}\{p_i(1 - p_i)\}.$$

The Hessian does not involve y, so $I(\beta) = -\nabla^2\ell$: Newton and Fisher scoring coincide. The update rearranges into

$$\beta^{(t+1)} = (X^\top W^{(t)} X)^{-1} X^\top W^{(t)} z^{(t)}, \qquad z^{(t)} = X\beta^{(t)} + s(W^{(t)})^{-1}(y - p^{(t)}),$$

a weighted least squares fit to the **working responses** $z^{(t)}$: **iteratively reweighted least squares** (IRLS). Any weighted least squares solver therefore fits logistic regression.

Example 7.2 (Poisson regression)

$y_i \sim \text{Poi}(\lambda_i)$ with $\log \lambda_i = x_i^\top \beta$: $\ell(\beta) = \sum_i \{y_i x_i^\top \beta - e^{x_i^\top \beta} - \log y_i!\}$, $\nabla \ell = X^\top (y - \lambda)$, $-\nabla^2 \ell = X^\top W X$ with $W = \text{diag}(\lambda_i)$. Again the Hessian is free of y, and Newton = Fisher scoring = IRLS with $z = X\beta + W^{-1}(y - \lambda)$.

7.5 Generalized linear models

These coincidences are no accident. An **exponential dispersion family** has density

$$p(y \mid \eta, \phi) = \exp\Big\{\frac{y\eta - b(\eta)}{a(\phi)} + c(y, \phi)\Big\},$$

with natural parameter η, mean $\mu = b'(\eta)$ and variance $b''(\eta)a(\phi)$. A **GLM** links the mean to a linear predictor, $g(\mu) = x^\top \beta$; the **canonical link** is $g = (b')^{-1}$, which makes $\eta = x^\top \beta$.

family	canonical link	variance function
normal	$\eta = \mu$	1
Poisson	$\eta = \log \mu$	μ
binomial	$\eta = \log \frac{\mu}{1-\mu}$	$\mu(1-\mu)$
gamma	$\eta = \mu^{-1}$	μ^2
inverse Gaussian	$\eta = \mu^{-2}$	μ^3

Proposition 7.2

With $a(\phi) \equiv 1$, $\mu_i = g^{-1}(x_i^\top \beta)$ and $\sigma_i^2 = b''(\eta_i)$:

$$\nabla \ell(\beta) = \sum_i \frac{(y_i - \mu_i)}{\sigma_i^2 g'(\mu_i)} x_i, \qquad I(\beta) = \sum_i \frac{1}{\sigma_i^2 g'(\mu_i)^2} x_i x_i^\top = X^\top W X.$$

Fisher scoring is IRLS with weights $w_i = 1/\{\sigma_i^2 g'(\mu_i)^2\}$. For the canonical link, $-\nabla^2 \ell = I(\beta)$, so Newton = Fisher scoring = IRLS and $-\ell$ is convex.

Proof. $\ell_i = y_i \eta_i - b(\eta_i)$, so $\partial \ell_i / \partial \eta_i = y_i - \mu_i$. By the chain rule, $d\mu = b''(\eta) d\eta = \sigma^2 d\eta$ and $d\mu = x^\top d\beta / g'(\mu)$, so $\partial \eta_i / \partial \beta = x_i / \{\sigma_i^2 g'(\mu_i)\}$, which gives the score. Differentiating again, one term is $-x_i x_i^\top / \{\sigma_i^2 g'(\mu_i)^2\}$ and the other is proportional to $(y_i - \mu_i)$, which has mean 0; this gives $I(\beta)$. For the canonical link $\eta_i = x_i^\top \beta$, so $\partial \eta_i / \partial \beta = x_i$ and $\nabla^2 \ell = -\sum_i b''(\eta_i) x_i x_i^\top$, which involves no y_i and equals $-I(\beta)$ (here $\sigma_i^2 g'(\mu_i) = 1$). ∎

For non-canonical links (e.g. probit) Fisher scoring and Newton differ and the likelihood need not be concave (probit happens to be log-concave). R's `glm` uses Fisher scoring.

Example 7.3 (Probit regression)

Let $y_i \sim \text{Ber}(p_i)$ with $p_i = \Phi(\eta_i)$, $\eta_i = \beta_0 + \beta_1 x_i$, and write $\Phi_i = \Phi(\eta_i)$, $\phi_i = \phi(\eta_i)$, $\mathbf{x}_i = (1, x_i)^\top$. Then

$$\ell(\beta) = \sum_i \{y_i \log \Phi_i + (1 - y_i) \log(1 - \Phi_i)\}, \qquad \nabla \ell = \sum_i \frac{(y_i - \Phi_i)\phi_i}{\Phi_i(1 - \Phi_i)} \mathbf{x}_i.$$

Differentiating ϕ_i / Φ_i and $-\phi_i / (1 - \Phi_i)$ in η with $\phi'(z) = -z\phi(z)$ gives

$$\nabla^2 \ell = \sum_i f_i \, \mathbf{x}_i \mathbf{x}_i^\top, \qquad f_i = -y_i \frac{\phi_i(\eta_i \Phi_i + \phi_i)}{\Phi_i^2} - (1 - y_i) \frac{\phi_i \{\phi_i - \eta_i(1 - \Phi_i)\}}{(1 - \Phi_i)^2},$$

which depends on y_i: the link is not canonical. Taking expectations ($Ey_i = \Phi_i$) gives the Fisher information $X^\top W X$ with $w_i = \phi_i^2/\{\Phi_i(1-\Phi_i)\}$, in agreement with Proposition 7.2 ($\sigma_i^2 = \Phi_i(1-\Phi_i)$, $g'(\mu_i) = 1/\phi_i$). The IRLS step is

$$\beta^{(t+1)} = (X^\top W^{(t)} X)^{-1} X^\top W^{(t)} z^{(t)}, \qquad z_i^{(t)} = \eta_i^{(t)} + \frac{y_i - \Phi_i^{(t)}}{\phi_i^{(t)}}.$$

7.6 Nonlinear least squares: Gauss–Newton

Gauss located the dwarf planet Ceres in 1801 by least squares. In general, minimize

$$f(\beta) = \tfrac{1}{2}\sum_{i=1}^{n}\{y_i - \mu_i(\beta)\}^2,$$

e.g. a logistic growth curve $\mu(x;\beta) = \beta_3/(1+e^{-\beta_1-\beta_2 x})$. With $J(\beta)$ the $n \times p$ Jacobian whose rows are $\nabla\mu_i(\beta)^\top$,

$$\nabla f = -J^\top(y-\mu), \qquad \nabla^2 f = J^\top J - \sum_i (y_i - \mu_i)\nabla^2\mu_i.$$

Gauss–Newton drops the second term, using the positive semidefinite $J^\top J$:

$$\beta^{(t+1)} = \beta^{(t)} + s(J^\top J)^{-1} J^\top (y - \mu(\beta^{(t)})),$$

i.e. each step is a linear least squares fit of the residuals on J. It is justified when residuals are small or μ is nearly linear, and it is exactly Fisher scoring for the model $y_i \sim N(\mu_i(\beta), \sigma^2)$, since $E[-\nabla^2\ell] \propto J^\top J$. (Gauss introduced the normal distribution partly to justify least squares.)

7.7 Gradient descent

$$x^{(t+1)} = x^{(t)} - \gamma_t \nabla f(x^{(t)}).$$

Minimizing the linear model $\nabla f^\top \Delta x$ over $\|\Delta x\|_2 \le 1$ gives $\Delta x = -\nabla f/\|\nabla f\|$: the steepest descent direction. Because a linear model is unbounded below, a step size must always be chosen. Each iteration is cheap and needs no Hessian, which makes the method attractive at large scale; but it zigzags and converges slowly, and it does not apply to nonsmooth f.

Theorem 7.3

Let f be convex and differentiable with L-Lipschitz gradient, $\|\nabla f(x) - \nabla f(y)\|_2 \le L\|x-y\|_2$, and let f attain its minimum $p^\star$ at $x^\star$. With constant step $\gamma \in (0, 1/L]$,

$$f(x^{(t)}) - p^\star \le \frac{\|x^{(0)} - x^\star\|_2^2}{2\gamma t}.$$

Proof. The Lipschitz gradient gives the quadratic upper bound $f(y) \le f(x) + \nabla f(x)^\top(y-x) + \frac{L}{2}\|y-x\|^2$. With $y = x^+ = x - \gamma\nabla f(x)$ and $\gamma L \le 1$, $f(x^+) \le f(x) - \frac{\gamma}{2}\|\nabla f(x)\|^2$ (descent). By convexity, $f(x) \le p^\star + \nabla f(x)^\top(x - x^\star)$, so

$$f(x^+) - p^\star \le \nabla f(x)^\top(x-x^\star) - \frac{\gamma}{2}\|\nabla f(x)\|^2 = \frac{1}{2\gamma}(\|x-x^\star\|^2 - \|x^+ - x^\star\|^2).$$

Summing over t iterations telescopes: $\sum_{k=1}^{t}\{f(x^{(k)}) - p^\star\} \le \frac{1}{2\gamma}\|x^{(0)} - x^\star\|^2$. Since $f(x^{(k)})$ is nonincreasing, the left side is at least $t\{f(x^{(t)}) - p^\star\}$. ∎

This $O(1/t)$ rate is *sublinear*; with strong convexity one gets linear convergence, but never Newton's quadratic rate.

Example 7.4

- Least squares $f(\beta) = \frac{1}{2}\|X\beta - y\|^2$: $\nabla f = X^\top X\beta - X^\top y$ is Lipschitz with $L = \sigma_{\max}(X)^2$, so $\gamma \le 1/\sigma_{\max}(X)^2$ works.
- Logistic regression: $\nabla f = -X^\top(y - p)$ is Lipschitz with $L = \frac{1}{4}\|X^\top X\|_2$, since $p(1-p) \le \frac{1}{4}$. The minimum may not exist (e.g. separable data); a ridge penalty $\frac{\rho}{2}\|\beta\|^2$ fixes this.
- Poisson regression: the gradient $-X^\top(y - e^{X\beta})$ is not globally Lipschitz, so a constant step size can fail; line search is needed. Indeed, an L-Lipschitz gradient forces $v^\top \nabla^2 f(\beta) v \le L\|v\|^2$ for all β, v. But for any v with $x_j^\top v > 0$ for some j,

$$v^\top \nabla^2 f(tv) v = \sum_i e^{t x_i^\top v} (x_i^\top v)^2 \ge e^{t x_j^\top v} (x_j^\top v)^2 \to \infty \quad (t \to \infty).$$

Example 7.5 (Multinomial logistic regression)

With K classes, dummy-coded responses $y_i \in \{0,1\}^K$, and $\log(p_{ik}/p_{iK}) = x_i^\top \beta_k$ ($k < K$, $\beta_K = 0$), we have $p_{ik} = e^{x_i^\top \beta_k}/(1 + \sum_{l<K} e^{x_i^\top \beta_l})$. With $B = [\beta_1, \dots, \beta_{K-1}] \in \mathbb{R}^{p\times(K-1)}$, Y_{-K} the first $K-1$ columns of Y, and P the $n \times (K-1)$ matrix of the p_{ik},

$$\ell(B) = \sum_i \sum_k y_{ik} \log p_{ik} = \operatorname{tr}(Y_{-K}^\top X B) - \sum_i \log\Big(1 + \sum_{l<K} e^{x_i^\top \beta_l}\Big),$$

$$\nabla_{\beta_k} \ell = \sum_i x_i (y_{ik} - p_{ik}), \qquad \text{so} \qquad B^{(t+1)} = B^{(t)} + \gamma X^\top (Y_{-K} - P^{(t)})$$

is gradient ascent. For $K = 2$ it reduces to logistic regression. Each step costs $O(npK)$ and needs no $p(K-1)$-dimensional Hessian; the gradient is Lipschitz (the Hessian is bounded by $\frac{1}{2}\|X\|_2^2$ in operator norm), so a constant step works.

Example 7.6 (Quantiles by optimization)

The α-quantile of X (cdf F, positive density f) solves $F(x) = \alpha$. Newton's method is $x^{(t+1)} = x^{(t)} - \{F(x^{(t)}) - \alpha\}/f(x^{(t)})$. It is Newton's method for minimizing a function ψ with $\psi' = F - \alpha$ and $\psi'' = f > 0$. Since $F(x) = E[\mathbf{1}\{X \le x\}]$, integrating in x and interchanging with the expectation gives

$$\psi(x) = E\big[(x - X)\mathbf{1}\{X \le x\}\big] - \alpha x = E\big[\rho_\alpha(X - x)\big] + \text{const}, \qquad \rho_\alpha(u) = u\{\alpha - \mathbf{1}(u < 0)\},$$

the expected **check loss** of quantile regression; it is convex. Gradient descent reads $x^{(t+1)} = x^{(t)} - \gamma_t\{F(x^{(t)}) - \alpha\}$ and needs no density. If F is unavailable but X is easy to sample, replace the gradient by its unbiased one-sample estimate,

$$x^{(t+1)} = x^{(t)} - \gamma_t(\mathbf{1}\{X_t \le x^{(t)}\} - \alpha), \qquad X_t \overset{\text{iid}}{\sim} F,$$

with $\sum_t \gamma_t = \infty$, $\sum_t \gamma_t^2 < \infty$ (e.g. $\gamma_t = c/t$): this *stochastic gradient* (Robbins–Monro) algorithm converges to the quantile almost surely. The simplest Monte Carlo alternative is the sample α-quantile of N draws, which has standard error $\sqrt{\alpha(1-\alpha)/N}/f(x_\alpha)$.

7.8 Coordinate descent

Minimize over one coordinate (or block) at a time, cycling:

$$x_j^{(t+1)} = \arg\min_{x_j} f(x_1^{(t+1)}, \ldots, x_{j-1}^{(t+1)}, x_j, x_{j+1}^{(t)}, \ldots, x_d^{(t)}).$$

Each one-dimensional problem is often easy (sometimes closed form). The objective values decrease monotonically, so they converge. It is the optimization analogue of Gauss–Seidel. But even for convex f, if f is not differentiable, coordinate descent can get stuck at a non-optimal point where no single coordinate move helps (a "corner" of a nonsmooth convex function).

Lecture 8. Numerical Integration

We want $\int_a^b f(x)\,dx$. Integrals are everywhere in statistics but rarely have closed forms:

- expectations, $E\sin X = \int \sin(x) f_X(x)\,dx$;
- Bayesian normalizing constants $\int f(x \mid \theta)\pi(\theta)\,d\theta$;
- likelihoods of mixed models, e.g. a Poisson GLM with random intercept, $\prod_i \int \prod_j f(y_{ij} \mid \gamma_i)\phi(\gamma_i)\,d\gamma_i$.

Two approaches: quadrature (now) and Monte Carlo (Lecture 10).

8.1 Newton–Cotes rules

Split $[a, b]$ into n intervals of length $h = (b-a)/n$, $x_i = a + ih$, approximate f on each $[x_i, x_{i+1}]$ by an interpolating polynomial of degree m, integrate the polynomial exactly, and add up.

- $m = 0$, **Riemann**: $\int_{x_i}^{x_{i+1}} f \approx h f(x_i)$.
- $m = 1$, **trapezoidal**: $\int_{x_i}^{x_{i+1}} f \approx \frac{h}{2}\{f(x_i) + f(x_{i+1})\}$, so $T(n) = h\{\frac{1}{2}f(a) + f(x_1) + \cdots + f(x_{n-1}) + \frac{1}{2}f(b)\}$.
- $m = 2$, **Simpson**: $\int_{x_i}^{x_{i+1}} f \approx \frac{h}{6}\{f(x_i) + 4f(\frac{x_i + x_{i+1}}{2}) + f(x_{i+1})\}$.

Proposition 8.1 (Accuracy)

For smooth f, the total errors are $O(n^{-1})$ (Riemann), $O(n^{-2})$ (trapezoidal) and $O(n^{-4})$ (Simpson). More precisely, per subinterval the local errors are

$$\frac{h^2}{2}f'(x_i) + O(h^3), \qquad -\frac{h^3}{12}f''(x_i) + O(h^4), \qquad -\frac{h^5}{2880}f^{(4)}(x_i) + O(h^6).$$

Proof. Integrate the Taylor expansion $f(x) = f(x_i) + (x - x_i)f'(x_i) + \frac{1}{2}(x - x_i)^2 f''(x_i) + \ldots$ over $[x_i, x_i + h]$:

$$\int_{x_i}^{x_{i+1}} f = h f(x_i) + \frac{h^2}{2}f'(x_i) + \frac{h^3}{6}f''(x_i) + \frac{h^4}{24}f^{(3)}(x_i) + \frac{h^5}{120}f^{(4)}(x_i) + O(h^6).$$

Riemann: the error is $\frac{h^2}{2}f'(x_i) + O(h^3)$. *Trapezoid*: expanding $f(x_{i+1})$ gives $\frac{h}{2}\{f(x_i) + f(x_{i+1})\} = hf(x_i) + \frac{h^2}{2}f' + \frac{h^3}{4}f'' + O(h^4)$, so the error is $(\frac{1}{6} - \frac{1}{4})h^3 f'' + O(h^4) = -\frac{h^3}{12}f''(x_i) + O(h^4)$. *Simpson*: expanding f at the midpoint and endpoint, the rule matches the terms through h^4 and differs at h^5 by $(\frac{1}{120} - \frac{5}{576})h^5 f^{(4)} = -\frac{h^5}{2880}f^{(4)}$. Summing n local errors multiplies by $n = (b-a)/h$, giving global orders h, h^2, h^4. ∎

Simpson uses about twice as many function evaluations per interval, so a fair comparison fixes the total number of evaluations; Simpson still wins by orders of magnitude for smooth f. For $\int_0^2 e^{-x}dx = 1 - e^{-2} \approx 0.864665$, with $n = 10$ the three rules have errors 8.9×10^{-2}, 2.9×10^{-3} and 4.8×10^{-7}, and on a log–log plot of error against n the slopes are -1, -2 and -4.

Example 8.1 (An infinite range)

Newton–Cotes rules need a finite interval. For $\int_1^\infty e^{-x}x^{-1/2}dx$ substitute $t = 1/x$, $dx =$

$-dt/t^2$:

$$\int_1^\infty e^{-x}x^{-1/2}dx = \int_0^1 e^{-1/t}t^{-3/2}\,dt.$$

The new integrand cannot be evaluated at $t = 0$ (it is $0\cdot\infty$ in floating point, giving `NaN`), so the trapezoidal and Simpson rules break at the endpoint. But $e^{-1/t}t^{-3/2} \to 0$ as $t \downarrow 0$, together with all its derivatives, so defining it as 0 at $t = 0$ gives a smooth integrand on $[0, 1]$ and the rules converge at their usual rates, to $\sqrt{\pi}\,\mathrm{erfc}(1) = \Gamma(\frac{1}{2}, 1) \approx 0.278806$. (Alternatively, use a rule that does not evaluate the endpoints, such as the midpoint rule or Gaussian quadrature.)

General m: interpolation. With $m+1$ distinct nodes $x_0, \ldots, x_m$, the **Lagrange polynomials** $L_j(x) = \prod_{k\neq j} \frac{x-x_k}{x_j-x_k}$ satisfy $L_j(x_k) = \delta_{jk}$, so $p(x) = \sum_j y_j L_j(x)$ interpolates (x_j, y_j).

Theorem 8.2 (Polynomial interpolation)

For $m + 1$ distinct points there is a unique polynomial of degree at most m through $(x_0, y_0), \ldots, (x_m, y_m)$.

Proof. Existence is the Lagrange form. If q is another, $p - q$ has degree $\leq m$ and $m + 1$ roots, hence is identically 0. ■

Corollary 8.3

If f is a polynomial of degree at most m, then $\int_a^b f = \sum_j w_j f(x_j)$ with $w_j = \int_a^b L_j(x)\,dx$.

Proof. By uniqueness, $f = \sum_j f(x_j)L_j$; integrate. ■

8.2 Gaussian quadrature

Newton–Cotes fixes equally spaced nodes and chooses weights to integrate polynomials of degree m exactly. Choosing the nodes as well gains much more.

Polynomials of degree $\leq m$ on (a, b) form a vector space $\mathcal{P}^m$ with inner product

$$\langle p, q\rangle = \int_a^b p(x)q(x)w(x)\,dx$$

for a positive weight w (here $w \equiv 1$). Gram–Schmidt applied to $1, x, x^2, \ldots$ produces orthonormal polynomials $q_0, q_1, \ldots,$ with q_k of exact degree k and orthogonal to all of $\mathcal{P}^{k-1}$; for $w \equiv 1$ on $(-1, 1)$ these are the **Legendre polynomials**.

Lemma 8.4

The roots of q_k are real, simple and lie in (a, b).

Proof. Let $r_1 < \cdots < r_l$ be the points in (a, b) where q_k changes sign. If $l < k$, then $q_k(x)\prod_{i\leq l}(x-r_i)$ does not change sign on (a, b) and is not identically zero, so its integral is nonzero. But $\prod_{i\leq l}(x-r_i)$ has degree $l < k$ and is orthogonal to q_k: a contradiction. So q_k has k sign changes, hence k distinct real roots in (a, b). ■

Theorem 8.5 (Gaussian quadrature)

Let $x_0, \dots, x_m$ be the roots of q_{m+1} and $w_j = \int_a^b L_j(x)w(x)\,dx$. Then

$$\int_a^b f(x)w(x)\,dx = \sum_{j=0}^{m} w_j f(x_j)$$

for every polynomial f of degree at most $2m+1$, and all $w_j > 0$. Degree $2m+1$ cannot be improved.

Proof. Divide: $f = gq_{m+1} + r$ with $g, r \in \mathcal{P}^m$. Then $\int gq_{m+1}w = \langle g, q_{m+1}\rangle = 0$, so $\int fw = \int rw$. At the nodes $q_{m+1}(x_j) = 0$, so $f(x_j) = r(x_j)$, and by Corollary 8.3 (with weight w) $\int rw = \sum w_j r(x_j) = \sum w_j f(x_j)$. Positivity: L_k^2 has degree $2m$, so $0 < \int L_k^2 w = \sum_j w_j L_k(x_j)^2 = w_k$. Optimality: $f = \prod_j (x - x_j)^2$ has degree $2m+2$ and $\int fw > 0$, while the rule gives 0. ■

So $m+1$ well-chosen nodes integrate polynomials of degree $2m+1$ exactly, against degree m for equally spaced nodes.

Weight functions. Changing the weight gives different families of orthogonal polynomials, suited to different integrals $\int g(x)w(x)\,dx$:

polynomials	weight $w(x)$	domain	related distribution
Legendre	1	$(-1, 1)$	uniform
Laguerre	$x^\alpha e^{-x}$	$(0, \infty)$	gamma
Hermite	e^{-x^2}	$(-\infty, \infty)$	normal
Jacobi	$(1-x)^\alpha(1+x)^\beta$	$(-1, 1)$	beta (after rescaling)

Each family satisfies a three-term recursion $q_m(x) = (\alpha_m + \beta_m x)q_{m-1}(x) - \gamma_m q_{m-2}(x)$, and its roots *interlace* (the roots of q_{m+1} separate those of q_m). The nodes are computed by Newton's method using these properties (or as eigenvalues of a tridiagonal matrix).

Gauss–Hermite quadrature. $\int f(x)\,dx = \int \{f(x)e^{x^2}\}e^{-x^2}dx \approx \sum_j w_j f(x_j)e^{x_j^2}$. It works well only if $f(x)e^{x^2}$ is close to a polynomial. For $f(x) = e^{-(x-\mu)^2/(2\sigma^2)}$ with μ far from 0 or σ^2 far from $\frac{1}{2}$, the nodes sit where f has no mass and the rule fails. The remedy is to *centre and scale*: substituting $z = (x-\mu)/(\sqrt{2}\sigma)$ gives $\sqrt{2}\sigma \int e^{-z^2}dz$, which a single node integrates exactly.

Example 8.2 (Binomial–logit posterior)

$x \sim B(n, p)$ with $p = e^\theta/(1+e^\theta)$ and prior $\theta \sim N(0, \sigma^2)$. The posterior normalizing constant is $\int \bar{g}(\theta)\,d\theta$ with $\bar{g}(\theta) = e^{\theta x}(1+e^\theta)^{-n}e^{-\theta^2/(2\sigma^2)}$. Applying Gauss–Hermite after only rescaling by σ performs badly when $x/n = 0.9$, because the integrand peaks near $\theta = \log 9 = 2.2$, far from the nodes. Centring at the maximizer θ^* of $\bar{g}$ and writing $\int \bar{g} = \int \{\bar{g}(z+\theta^*)e^{z^2}\}e^{-z^2}dz$ gives an accurate rule with few nodes.

Example 8.3 (Poisson posterior: centre *and* scale)

Let $y_1, \dots, y_{10}$ be Poisson with mean e^α, prior $\alpha \sim N(0, 100)$, and data $11, 19, 27, 12, 14, 11, 10, 13, 15, 10$ ($\sum y_i = 142$). The log unnormalized posterior is $h(\alpha) = 142\alpha - 10e^\alpha - \alpha^2/200$. Its mode $\hat{\alpha} = 2.65306$ solves $h' = 0$ (by Newton or bisection), and the

curvature gives the scale $\hat{s} = \{-h''(\hat{\alpha})\}^{-1/2} = 0.08392$. Substituting $\alpha = \hat{\alpha} + \sqrt{2}\hat{s}z$,

$$E[g(\alpha) \mid y] = \frac{\int g(\hat{\alpha} + \sqrt{2}\hat{s}z) e^{h(\hat{\alpha}+\sqrt{2}\hat{s}z) - h(\hat{\alpha}) + z^2} e^{-z^2} dz}{\int e^{h(\hat{\alpha}+\sqrt{2}\hat{s}z) - h(\hat{\alpha}) + z^2} e^{-z^2} dz},$$

and both integrals go to Gauss–Hermite. (The constants $\sqrt{2}\hat{s}$ cancel; subtracting $h(\hat{\alpha})$ prevents overflow of e^h, since $h(\hat{\alpha}) \approx 235$.) Since the posterior is nearly normal, the bracketed functions are nearly constant or polynomial:

nodes	posterior mean	posterior variance
2	2.651881	0.0070417
5	2.649534	0.0070677
10	2.6495297	0.00706793
15	2.649529692	0.007067929

Fifteen nodes reproduce adaptive numerical integration to 10 significant digits. Without centring, the nodes of e^{-z^2} sit around 0, where the posterior (concentrated near 2.65 with standard deviation 0.084) is numerically zero, and the rule fails completely.

Example 8.4 (Variance of the sample median)

For $X_1, \ldots, X_n$ i.i.d. $N(0,1)$, $n = 2m+1$, the median $X_{(m+1)}$ has density $n\binom{n-1}{m}\Phi(x)^m\{1-\Phi(x)\}^m\phi(x)$: one observation falls in $[x, x+dx)$ (n choices, probability $\phi(x)dx$), and exactly m of the other $n-1$ fall below x. The density is symmetric, so $EX_{(m+1)} = 0$ and

$$\begin{aligned}\operatorname{Var} X_{(m+1)} &= \int_{-\infty}^{\infty} x^2 n \binom{n-1}{m} \Phi(x)^m \{1-\Phi(x)\}^m \phi(x)\, dx \\ &= \frac{2n}{\sqrt{\pi}} \binom{n-1}{m} \int u^2 \Phi(\sqrt{2}u)^m \{1-\Phi(\sqrt{2}u)\}^m e^{-u^2} du,\end{aligned}$$

after $x = \sqrt{2}u$. Gauss–Hermite with k nodes gives:

k	2	4	8	16	32	64	exact	$\frac{\pi}{2n}$
$n = 11$	0.117	0.238	0.234	0.161	0.1379	0.13716	0.13716	0.1428
$n = 21$	0.007	0.057	0.127	0.123	0.0840	0.07349	0.07326	0.0748

Convergence is slow and erratic because $\Phi^m(1-\Phi)^m$ is sharply peaked, with width of order $n^{-1/2}$, while the Hermite nodes are spread for a unit-variance shape. Rescaling by the asymptotic standard deviation, as in the previous example, fixes this. The asymptotic variance $1/\{4n\phi(0)^2\} = \pi/(2n)$ overstates the exact value by 4.1% for $n = 11$ and 2.1% for $n = 21$.

Example 8.5 (Generalized Gauss–Laguerre)

With $x = u^2$, $\int_0^\infty \sqrt{x}e^{-x}dx = 2\int_0^\infty u^2 e^{-u^2}du = \frac{\sqrt{\pi}}{2}$ $(= \Gamma(\frac{3}{2}))$. To compute $\int_0^\infty f(x)\sqrt{x}e^{-x}dx$, the natural rule is Gauss quadrature with weight $w(x) = x^{1/2}e^{-x}$ (generalized Laguerre, $\alpha = \frac{1}{2}$), whose nodes are the roots of the corresponding orthogonal polynomial. For $f \equiv 1$ a single node ($m = 0$, weight $w_0 = \Gamma(\frac{3}{2})$) is already exact. For $f = \log x$, with exact value $\Gamma'(\frac{3}{2}) = \frac{\sqrt{\pi}}{2}(2 - \gamma - 2\log 2) \approx 0.032338$, two sources of error remain.

(i) *Method error*: $\log x$ is not a polynomial and is singular at 0, so the rule is not exact and the error decreases only slowly (algebraically, not geometrically) with m.

(ii) *Floating-point error*: the nodes and weights are themselves computed approximately (by Newton's method or an eigenvalue problem), the smallest node is close to 0 where $\log x$ is large, the largest weights are tiny and may underflow for large m, and the result is small (0.03) compared with the terms being summed, so cancellation between positive and negative terms magnifies rounding errors.

8.3 Laplace approximation

For integrals $\int f(x)e^{-ag(x)}dx$ with large a, where g has a unique minimum at 0, the integrand concentrates near 0, and $g(x) \approx g(0) + \frac{1}{2}g''(0)x^2$ gives

$$\int f(x)e^{-ag(x)}dx \approx f(0)e^{-ag(0)}\sqrt{\frac{2\pi}{ag''(0)}}:$$

integration is replaced by optimization.

Theorem 8.6 (Laplace)

Suppose (1) for every $\delta > 0$ there is $\rho > 0$ with $g(x) - g(0) \geq \rho$ for $|x| \geq \delta$;
(2) g is twice continuously differentiable near 0 with $g'(0) = 0$, $g''(0) > 0$;
(3) f is continuous near 0 with $f(0) > 0$;
(4) $\int |f|e^{-ag} < \infty$ for $a \geq a_0$. Then

$$\lim_{a\to\infty} \frac{\int f(x)e^{-ag(x)}dx}{f(0)e^{-ag(0)}\sqrt{2\pi/(ag''(0))}} = 1.$$

Proof. Take $g(0) = 0$. Given $\epsilon \in (0, \frac{1}{2}g''(0))$, choose δ such that for $|x| < \delta$,

$$(1-\epsilon)f(0) \leq f(x) \leq (1+\epsilon)f(0), \qquad \tfrac{1}{2}(g''(0)-2\epsilon)x^2 \leq g(x) \leq \tfrac{1}{2}(g''(0)+2\epsilon)x^2.$$

Tails. For $|x| \geq \delta$, $g \geq \rho$, so for $a \geq a_0$,

$$\int_{|x|\geq\delta} |f|e^{-ag} = \int_{|x|\geq\delta} |f|e^{-a_0 g}e^{-(a-a_0)g} \leq e^{-(a-a_0)\rho}\int |f|e^{-a_0 g} = O(e^{-\rho a}).$$

Centre, upper bound. $\int_{|x|<\delta} fe^{-ag} \leq (1+\epsilon)f(0)\int e^{-\frac{a}{2}(g''(0)-2\epsilon)x^2}dx = (1+\epsilon)f(0)\sqrt{\frac{2\pi}{a(g''(0)-2\epsilon)}}$. *Centre, lower bound.* $\int_{|x|<\delta} fe^{-ag} \geq (1-\epsilon)f(0)\int_{|x|<\delta} e^{-\frac{a}{2}(g''(0)+2\epsilon)x^2}dx$, and the Gaussian integral over $|x| \geq \delta$ is $O(e^{-\lambda a})$ with $\lambda = \frac{1}{2}(g''(0)+2\epsilon)\delta^2$. Multiplying by $\sqrt{a}$ and letting $a \to \infty$,

$$(1-\epsilon)f(0)\sqrt{\frac{2\pi}{g''(0)+2\epsilon}} \leq \liminf \sqrt{a}\int fe^{-ag} \leq \limsup \sqrt{a}\int fe^{-ag} \leq (1+\epsilon)f(0)\sqrt{\frac{2\pi}{g''(0)-2\epsilon}}.$$

Let $\epsilon \to 0$. ■

Example 8.6 (Stirling's formula)

With $x = tz$, $\Gamma(t+1) = \int_0^\infty x^t e^{-x}dx = t^{t+1}\int_0^\infty e^{-t(z-\log z)}dz$. Here $g(z) = z - \log z$ is minimized at $z = 1$ with $g(1) = 1$, $g''(1) = 1$, so $\Gamma(t+1) \sim \sqrt{2\pi}\, t^{t+1/2}e^{-t}$.

Posterior expectations. With log-likelihood $\ell_n(\theta)$, log-prior $\pi(\theta)$ and $\tilde{h} = \log h > 0$,

$$E[h(\Theta) \mid x] = \frac{\int e^{\tilde{h}+\ell_n+\pi} d\theta}{\int e^{\ell_n+\pi} d\theta}.$$

Apply Laplace to numerator and denominator separately, at the maximizers $\tilde{\theta}$ of $\tilde{h} + \ell_n + \pi$ and $\hat{\theta}$ of $\ell_n + \pi$ (the posterior mode):

$$E[h(\Theta) \mid x] \approx \exp\{(\tilde{h} + \ell_n + \pi)(\tilde{\theta}) - (\ell_n + \pi)(\hat{\theta})\} \sqrt{\frac{(\ell_n + \pi)''(\hat{\theta})}{(\tilde{h} + \ell_n + \pi)''(\tilde{\theta})}}.$$

For large n the log-posterior is sharply peaked, and the approximation is accurate (its relative error is $O(n^{-2})$, since the leading errors of numerator and denominator cancel).

Example 8.7 (Beta moments)

For $\Theta \sim \text{Beta}(\alpha, \beta)$ (a binomial posterior), $E\Theta^k = \frac{\alpha(\alpha+1)\cdots(\alpha+k-1)}{(\alpha+\beta)\cdots(\alpha+\beta+k-1)}$ exactly. With $g(\theta) = -(\alpha-1)\log\theta - (\beta-1)\log(1-\theta)$, minimized at $\hat{\theta} = \frac{\alpha-1}{\alpha+\beta-2}$,

$$\int_0^1 \theta^{\alpha-1}(1-\theta)^{\beta-1} d\theta \approx \Big(\frac{\alpha-1}{\alpha+\beta-2}\Big)^{\alpha-1} \Big(\frac{\beta-1}{\alpha+\beta-2}\Big)^{\beta-1} \sqrt{\frac{2\pi(\alpha-1)(\beta-1)}{(\alpha+\beta-2)^3}},$$

and the same with α replaced by $\alpha + k$ for the numerator. If $\alpha, \beta \to \infty$ with $\alpha/(\alpha+\beta) \to p$, both the exact moment and its approximation tend to p^k.

Unlike quadrature, whose number of nodes grows exponentially with dimension, the Laplace approximation extends directly to $\mathbb{R}^d$: $\int f e^{-ag} \approx f(\hat{x}) e^{-ag(\hat{x})} (2\pi/a)^{d/2} \det\{\nabla^2 g(\hat{x})\}^{-1/2}$.

Lecture 9. Random Number Generation

9.1 Uniform random numbers

All random number generation starts from i.i.d. Unif$(0,1)$ variates. Computers produce none: they produce deterministic **pseudo-random** sequences that only look random.

Multiplicative congruential generators. Take a large prime modulus m and a multiplier $a \in \{2,\dots,m-1\}$, and iterate $x_{k+1} = ax_k \bmod m$, outputting $u_k = x_k/m$.

Proposition 9.1

The map $f(x) = ax \bmod m$ is a bijection of $\{1,\dots,m-1\}$. The sequence is periodic with period $n = \min\{k \geq 1 : a^k \equiv 1 \bmod m\}$, which divides $m-1$.

Proof. $\gcd(a,m) = 1$ because m is prime, so ax is not a multiple of m for $1 \leq x \leq m-1$: f maps the set into itself. If $ax \equiv ay$, then $m \mid a(x-y)$, hence $m \mid x-y$ and $x = y$: f is injective, hence bijective on a finite set. Since $x_k = a^k x_0 \bmod m$, the sequence returns to x_0 exactly when $a^k \equiv 1$. By Fermat's little theorem $a^{m-1} \equiv 1 \bmod m$, so the period is at most $m-1$ (and divides it). ■

x_0 is the **seed**. If a is a **primitive root** modulo m (period $m-1$), the sequence runs through all of $\{1,\dots,m-1\}$. A classical choice is $m = 2^{31}-1$ (a Mersenne prime) and $a = 7^5 = 16807$, with period $2^{31}-2$ (used in early MATLAB).

Mixed congruential generators. $x_{i+1} = (ax_i + c) \bmod m$ with $c \neq 0$ allows $m = 2^M$, so that reduction modulo m is free in binary arithmetic. By the Hull–Dobell theorem the period is the full m iff $\gcd(c,m) = 1$, every prime factor of m divides $a-1$, and $4 \mid a-1$ when $4 \mid m$. Both $m = 2048$, $a = 1229$, $c = 1$ and the old C-library choice $m = 2^{32}$, $a = 1103515245$, $c = 12345$ satisfy this. But the first generator can produce only 2048 distinct values, multiples of $1/2048$, and repeats after 2048 draws, which is useless for simulation. The second has a long period, yet its low-order bits are poor: modulo 2^k the sequence has period only 2^k, so the last bit simply alternates. Plotting consecutive pairs (u_i, u_{i+1}) exposes the lattice structure of both.

A good generator needs a long period and output that passes statistical tests of uniformity and independence. Congruential generators have a structural flaw: consecutive p-tuples $(u_k,\dots,u_{k+p-1})$ lie on a small number of parallel hyperplanes in $[0,1]^p$ (Marsaglia's "random numbers fall mainly in the planes"; the notorious RANDU generator puts all triples on 15 planes). Shuffling helps. R's default, the **Mersenne Twister** (Matsumoto–Nishimura, 1998), has period $2^{19937}-1$ and is equidistributed in up to 623 dimensions.

9.2 The inverse CDF method

Motivation. If U is uniform, $F^{-1}(U)$ has cdf F. Whenever the quantile function is available in closed form, this is the simplest way to simulate from F.

For a cdf F define the generalized inverse $F^{-1}(u) = \inf\{x : F(x) \geq u\}$, $0 < u < 1$.

Theorem 9.2

Let $X \sim F$ and $U \sim$ Unif$(0,1)$.

(a) $F^{-1}(U)$ has cdf F.

(b) If F is continuous, $F(X) \sim \text{Unif}(0,1)$.

(c) In general, $P\{F(X) \le y\} \le y$ for $y \in [0,1]$.

Proof. (a) We show $\{F^{-1}(u) \le t\} = \{u \le F(t)\}$. The set $\{x : F(x) \ge u\}$ is an interval $[F^{-1}(u), \infty)$ that contains its left endpoint, by monotonicity and right continuity; so $F(F^{-1}(u)) \ge u$. If $F^{-1}(u) \le t$ then $u \le F(F^{-1}(u)) \le F(t)$. Conversely, if $u \le F(t)$ then t belongs to the set, so $F^{-1}(u) \le t$. Hence $P(F^{-1}(U) \le t) = P(U \le F(t)) = F(t)$.

(b) First, $P\{F(X) \le F(t)\} = F(t)$ for every t: the event equals $\{X \le t\} \cup \{X > t, F(X) = F(t)\}$, and the second part is a set where F is flat, which has probability 0. For $u \in (0,1)$, continuity gives t with $F(t) = u$, so $P\{F(X) \le u\} = u$.

(c) By (a), we may take $X = F^{-1}(U)$, and $F(F^{-1}(U)) \ge U$. Hence $\{F(X) \le y\} \subset \{U \le y\}$. ■

Example 9.1

- Exponential, $F(x) = 1 - e^{-\lambda x}$: $F^{-1}(u) = -\frac{1}{\lambda}\log(1-u)$, and since $1 - U \overset{d}{=} U$ one uses $-\frac{1}{\lambda}\log U$.
- Cauchy(μ, β): $F(x) = \frac{1}{2} + \frac{1}{\pi}\arctan\frac{x-\mu}{\beta}$, $F^{-1}(u) = \mu + \beta\tan(\pi u - \frac{\pi}{2})$.
- Discrete uniform on $\{1, \dots, k\}$: $F^{-1}(u) = \lceil ku \rceil$.
- Geometric(p) on $\{1, 2, \dots\}$: if $Y \sim \text{Exp}(\lambda)$ with $1 - e^{-\lambda} = p$, then $\lceil Y \rceil$ is geometric, so $X = \lceil \log U / \log(1-p) \rceil$.

When F^{-1} has no closed form but F (or only the density f) is computable, $F^{-1}(u)$ can still be found by solving $F(x) = u$ numerically for each u: bisection on a bracketing interval, or Newton's method $x \leftarrow x - \{F(x) - u\}/f(x)$ (Lecture 6), with $F(x) = \int_{-\infty}^{x} f$ evaluated by quadrature if necessary. This is exact up to the tolerance but costly per draw, and for heavy tails (e.g. t_4) the bracketing interval must be wide enough.

9.3 Normal random numbers

Φ^{-1} has no closed form (though accurate rational approximations exist).

Box–Muller. For independent $X, Y \sim N(0,1)$ in polar coordinates (R, Θ), the joint density is $\frac{1}{2\pi}e^{-r^2/2}r$, so $\Theta \sim \text{Unif}(0, 2\pi)$ independently of R, and $P(R^2 > s) = e^{-s/2}$, i.e. $R^2 \sim$ Exp with mean 2. Hence for independent $U, V \sim \text{Unif}(0,1)$,

$$R = \sqrt{-2\log U}, \quad \Theta = 2\pi V, \qquad (X, Y) = (R\cos\Theta, R\sin\Theta)$$

are independent standard normals.

Marsaglia's polar method. If (U, V) is uniform on the unit disk, written $(T\cos\Phi, T\sin\Phi)$, then $P(T^2 \le t, \Phi \le \phi) = \frac{\pi t \cdot \phi/(2\pi)}{\pi} = t\frac{\phi}{2\pi}$, so $T^2 \sim \text{Unif}(0,1)$ and $\Phi \sim \text{Unif}(0, 2\pi)$ independently. So $-2\log T^2$ plays the role of R^2, and $(U/T, V/T)$ that of $(\cos\Theta, \sin\Theta)$:

(1) draw U, V i.i.d. Unif$(-1, 1)$;

(2) if $T^2 = U^2 + V^2 \ge 1$, return to (1);

(3) output $X = U\sqrt{-2\log T^2/T^2}$ and $Y = V\sqrt{-2\log T^2/T^2}$.

It avoids trigonometric functions at the price of using $4/\pi$ times as many pairs (the rejection step is an instance of Section 9.5).

9.4 Generating from definitions

- Bernoulli(p): $\mathbf{1}\{U \le p\}$. Binomial(n, p): sum of n Bernoullis. Negative binomial(r, p): sum of r geometrics.
- Poisson(λ): the number N with $\prod_{i=1}^{N} U_i \ge e^{-\lambda} > \prod_{i=1}^{N+1} U_i$. Indeed $T_i = -\log U_i/\lambda$ are i.i.d. Exp(λ) interarrival times, and N is the number of arrivals of a rate-λ Poisson process in $[0, 1]$: $T_1 + \cdots + T_N \le 1 < T_1 + \cdots + T_{N+1}$.
- $\chi^2(\nu)$: $\sum_{i \le \nu} Z_i^2$; for even ν, also $-2 \log \prod_{i \le \nu/2} U_i$, since $\chi^2(\nu) = \text{Gamma}(\nu/2, 2)$.
- $t(\nu) = Z/\sqrt{X/\nu}$ and $F(\nu_1, \nu_2) = (X_1/\nu_1)/(X_2/\nu_2)$ from independent normal and chi-square variates.

9.5 Acceptance–rejection sampling

Motivation. When F^{-1} is not available but the density is known up to a constant, we can sample from an easier envelope density and accept each draw with probability proportional to the ratio; the accepted draws have exactly the target distribution.

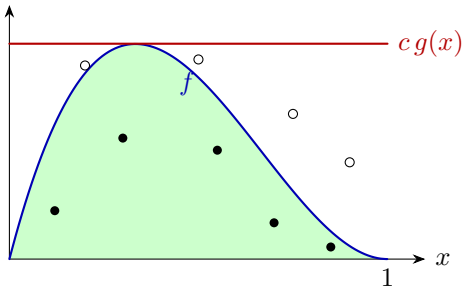


Figure. *Acceptance–rejection with target f (a Beta(2, 3) density) and uniform proposal g, $c = \sup f/g$: points uniform under $c\,g$ are kept if they fall under f (filled) and rejected otherwise (open). Accepted x-coordinates have density f; the acceptance rate is $1/c$.*

To sample from a density f with no tractable cdf (von Neumann):

(1) find a density g that is easy to sample and a constant c with $f \le cg =: h$ (an *envelope*);
(2) draw $X \sim g$ and an independent $U \sim \text{Unif}(0, 1)$;
(3) accept X if $U \le f(X)/h(X)$; otherwise return to (2).

The **body** of a nonnegative function f is $B_f = \{(x, y) : 0 \le y \le f(x)\}$, with area $|B_f| = \int f$.

Theorem 9.3

If $X \sim g$ and $U \sim \text{Unif}(0, 1)$ are independent, then $(X, cg(X)U)$ is uniform on B_{cg}.

Proof. Given $X = x$, $Y = cg(x)U$ is uniform on $[0, cg(x)]$, with density $\frac{1}{cg(x)}\mathbf{1}\{0 \le y \le cg(x)\}$. Multiplying by $g(x)$, the joint density is $\frac{1}{c}\mathbf{1}_{B_{cg}}(x, y)$; and $|B_{cg}| = c$. ■

Corollary 9.4

An accepted X has density f, and each proposal is accepted with probability $1/c$.

Proof. Acceptance is the event $(X, Y) \in B_f \subset B_{cg}$. Conditioning a uniform distribution on a subset gives the uniform distribution on the subset, so the accepted (X, Y) is uniform on B_f, with density $\mathbf{1}_{B_f}/|B_f|$. Its x-marginal is $\int_0^{f(x)} dy/|B_f| = f(x)/\int f = f(x)$. The acceptance probability is $|B_f|/|B_{cg}| = 1/c$. ■

So the envelope should be tight (c close to 1); the number of proposals per accepted draw is geometric with mean c. f need only be known up to a constant, as long as the envelope uses the same constant.

Example 9.2 (Uniform on the disk)

In Marsaglia's method, $f = \frac{1}{\pi}\mathbf{1}\{u^2 + v^2 < 1\}$ and $g = \frac{1}{4}\mathbf{1}_{[-1,1]^2}$, so $c = 4/\pi$ and $f/(cg)$ is the indicator of the disk: accept iff $U^2 + V^2 < 1$.

Example 9.3 (Beta variates from uniforms)

For Beta(a, b) with $a, b > 1$ the density is bounded by its value $d_{\max}$ at the mode $(a-1)/(a+b-2)$. Taking $g = \text{Unif}(0, 1)$ and $c = d_{\max}$: draw $Y \sim \text{Unif}(0, 1)$ and $U \sim \text{Unif}(0, 1)$, and keep Y iff $U \le f(Y)/d_{\max}$. The kept values are a Beta(a, b) sample and the acceptance rate is $1/d_{\max}$. Running this on 1000 proposals returns a random number (about $1000/d_{\max}$) of Beta draws. The method is simple but wasteful when the density is peaked (large a, b).

Example 9.4 (Gamma variates)

Since $c\,\text{Gamma}(\alpha, 1) = \text{Gamma}(\alpha, c)$ and $\text{Gamma}(\alpha, 1) \overset{d}{=} \text{Gamma}(\lfloor\alpha\rfloor, 1) \oplus \text{Gamma}(\alpha - \lfloor\alpha\rfloor, 1)$, where the first is a sum of exponentials, it suffices to handle $0 < \alpha < 1$. Then $x^{\alpha-1}e^{-x} \le x^{\alpha-1}$ on $[0, 1]$ and $\le e^{-x}$ on $(1, \infty)$, giving the envelope $h(x) = \{x^{\alpha-1}\mathbf{1}_{[0,1]} + e^{-x}\mathbf{1}_{(1,\infty)}\}/\Gamma(\alpha)$ with $c = (\frac{1}{\alpha} + \frac{1}{e})/\Gamma(\alpha)$. The proposal density $g = h/c$ has cdf $G(x) = x^\alpha/(1 + \alpha/e)$ on $[0, 1]$ and $1 - \alpha e^{-x}/(1 + \alpha/e)$ for $x > 1$, which inverts in closed form:

$$G^{-1}(u) = \begin{cases} \{(1 + \alpha/e)u\}^{1/\alpha}, & u \le 1/(1 + \alpha/e), \\ -\log\{(1/\alpha + 1/e)(1 - u)\}, & \text{otherwise.} \end{cases}$$

Example 9.5 (Gamma with $\alpha > 1$: a Cauchy envelope)

Let $\tilde{f}(x) = x^{\alpha-1}e^{-x}$ $(x > 0)$, $\alpha > 1$, $\mu = \alpha - 1$, $\sigma^2 = 2\alpha - 1$, and

$$g(x) = \frac{\mu^\mu e^{-\mu}}{1 + (x - \mu)^2/\sigma^2},$$

a multiple of the Cauchy(μ, σ) density. *Domination*: $k(x) = x^{\alpha-1}e^{-x}\{\sigma^2 + (x - \mu)^2\}$ has derivative $-x^{\alpha-2}e^{-x}(x - \alpha)^2(x - \mu)$, which is ≥ 0 for $x < \mu$ and ≤ 0 for $x > \mu$, so $k(x) \le k(\mu) = \sigma^2\mu^\mu e^{-\mu}$; dividing by $\sigma^2 + (x - \mu)^2$ gives $\tilde{f} \le g$ on $(0, \infty)$. *Algorithm*: draw $Y = \mu + \sigma\tan\{\pi(V - \frac{1}{2})\}$ with $V \sim \text{Unif}(0, 1)$ (inverse cdf); reject if $Y \le 0$; otherwise accept if $U \le \tilde{f}(Y)/g(Y)$. *Efficiency*: since $\int g = \pi\sigma\mu^\mu e^{-\mu}$ and $\int \tilde{f} = \Gamma(\alpha)$, the acceptance probability is

$$\frac{\Gamma(\alpha)}{\pi\sqrt{2\alpha - 1}\,(\alpha - 1)^{\alpha-1}e^{-(\alpha-1)}} \approx 0.47,\ 0.50,\ 0.54,\ 0.56 \quad (\alpha = 1.5, 2, 5, 100),$$

increasing to $1/\sqrt{\pi} \approx 0.564$ as $\alpha \to \infty$, by Stirling's formula $\Gamma(\alpha) = \Gamma(\mu + 1) \sim \sqrt{2\pi\mu}\,\mu^\mu e^{-\mu}$. So about two proposals per draw, uniformly in α, unlike the uniform envelope above.

9.6 Multivariate distributions

Batch methods.

- Multinomial$(n; p_1, \ldots, p_k)$: draw n categories, category j when $U \in [\sum_{l<j} p_l, \sum_{l\le j} p_l)$, and count. (Alternatively, independent Poissons with means λp_j conditioned on their sum being n, which is inefficient.)
- Multivariate normal $N(\mu, \Sigma)$: find L with $LL^\top = \Sigma$ and set $\mu + LZ$, $Z \sim N(0, I_p)$. Use the Cholesky factor (cheapest) or the symmetric square root $Q\Lambda^{1/2}Q^\top$ from the eigendecomposition (which also handles singular Σ). For large p the factorization is the bottleneck.
- Multivariate $t_\nu(\mu, \Sigma)$: $\mu + X/\sqrt{W/\nu}$ with $X \sim N(0, \Sigma)$ independent of $W \sim \chi^2(\nu)$.

Sequential (conditional) sampling. Draw components one at a time from their conditional distributions:

- Multinomial: $X_1 \sim B(n, p_1)$, then $X_2 \mid X_1 = x_1 \sim B(n - x_1, p_2/(1 - p_1))$, and so on.
- Multivariate normal: $X_1 \sim N(\mu_1, \sigma_{11})$, then $(X_2, \ldots, X_p) \mid X_1 = x_1 \sim N\big(\mu_2 + \Sigma_{21}(x_1 - \mu_1)/\sigma_{11},\ \Sigma_{22} - \Sigma_{21}\Sigma_{12}/\sigma_{11}\big)$, and recurse.

Lecture 10. Monte Carlo Integration

10.1 Naive Monte Carlo

To compute $\mu = \mathbb{E}f(X) = \int f(x)p(x)\,dx$ for a density p, draw $X_1, \dots, X_n$ i.i.d. from p and use

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} f(X_i).$$

It is unbiased; by the strong law $\hat{\mu}_n \to \mu$ almost surely when $\mathbb{E}|f(X)| < \infty$; and if $\sigma^2 = \operatorname{Var} f(X) < \infty$, the CLT gives $\sqrt{n}(\hat{\mu}_n - \mu) \Rightarrow N(0, \sigma^2)$. With the sample variance $\hat{\sigma}^2$ of the $f(X_i)$, an approximate 95% interval is $\hat{\mu}_n \pm 1.96\,\hat{\sigma}/\sqrt{n}$.

The error is $O_p(n^{-1/2})$ *regardless of the dimension*, while deterministic quadrature on a grid suffers from the curse of dimensionality. In one or two dimensions, quadrature (Lecture 8) is far more accurate; Monte Carlo wins in high dimensions. To improve on $n^{-1/2}$ one cannot do much, but the constant σ^2 can be reduced: this is **variance reduction**.

Example 10.1

- $\int_0^\infty \sqrt{x}e^{-x}dx = \Gamma(3/2) = \sqrt{\pi}/2$ is $\mathbb{E}\sqrt{X}$ for $X \sim \text{Exp}(1)$; the average of $\sqrt{X_i}$ estimates it with finite variance.
- $\int_0^1 x^{-1/2}dx = 2$ is $\mathbb{E}U^{-1/2}$, but $\mathbb{E}U^{-1} = \infty$: the estimator is consistent yet has infinite variance, so the CLT interval is meaningless and the running averages show occasional large jumps. The substitution $x = y^2$ turns the integral into $\int_0^1 2\,dy$, estimated exactly. A transformation that flattens the integrand is the cheapest variance reduction.
- Buffon's needle: a needle of length L dropped on a floor ruled with parallel lines at spacing $D \geq L$ crosses a line with probability $2L/(\pi D)$. The centre's distance to the nearest line is $Y \sim \text{Unif}(0, D/2)$ and the angle $\Theta \sim \text{Unif}(0, \pi/2)$, independent; the needle crosses iff $Y \leq \frac{L}{2}\sin\Theta$, which has probability $\frac{2}{D}\cdot\frac{2}{\pi}\int_0^{\pi/2}\frac{L}{2}\sin\theta\,d\theta = \frac{2L}{\pi D}$. The crossing frequency thus estimates π: with k crossings in n throws, $\hat{p} = k/n$ is unbiased for $p = 2L/(\pi D)$ and $\hat{\pi} = 2L/(D\hat{p}) = 2Ln/(Dk)$. But $\hat{\pi}$ is *not* unbiased: $1/x$ is convex, so $E[1/\hat{p}] > 1/p$ by Jensen, and in fact $E\hat{\pi} = \infty$ because $k = 0$ has positive probability. By the delta method with $g(p) = 2L/(Dp)$, $g'(p)^2 = \{2L/(Dp^2)\}^2 = \pi^2/p^2$, so

$$\operatorname{Var}\hat{\pi} \approx g'(p)^2\frac{p(1-p)}{n} = \frac{\pi^2(1-p)}{np},$$

which decreases in p; the best choice is $L = D$, giving $\operatorname{Var}\hat{\pi} \approx 5.63/n$. Even then, three correct digits need tens of millions of throws.

10.2 Importance sampling

For any density h with $h > 0$ wherever $fp \neq 0$,

$$\mu = \int f(x)\frac{p(x)}{h(x)}h(x)\,dx = \mathbb{E}_h\Big[f(Y)\frac{p(Y)}{h(Y)}\Big], \qquad \hat{\mu}_h = \frac{1}{n}\sum_{i=1}^{n} f(Y_i)w(Y_i), \quad Y_i \overset{\text{iid}}{\sim} h,$$

with **importance weights** $w = p/h$. It is unbiased with variance $\frac{1}{n}\{\int f^2p^2/h - \mu^2\}$. This is useful when p is hard to sample, or when p rarely visits the region where f is large.

Proposition 10.1

The variance of $\hat{\mu}_h$ is minimized by $h^\star \propto |f|\,p$. If $f \geq 0$, then $h^\star = fp/\mu$ and the variance is zero.

Proof. By Cauchy–Schwarz, for every admissible h,

$$\Big(\int |f|p\Big)^2 = \Big(\int \frac{|f|p}{\sqrt{h}}\sqrt{h}\Big)^2 \leq \int \frac{f^2p^2}{h}\int h = \int \frac{f^2p^2}{h},$$

with equality iff $|f|p/\sqrt{h} \propto \sqrt{h}$, i.e. $h \propto |f|p$. If $f \geq 0$, $fw \equiv \mu$ under $h^\star$. ■

$h^\star$ needs μ itself, so it is unusable, but it is a guide: choose h roughly proportional to $|f|p$, and with tails at least as heavy as fp; otherwise the weights explode and the variance may be infinite. A useful diagnostic is the spread of the weights.

Example 10.2 (Normal tail probability)

Estimate $\pi(c) = P(Z > c) = \Phi(-c)$ for large c. The naive estimator $\frac{1}{n}\sum \mathbf{1}\{Z_i > c\}$ has relative error $\sqrt{(1-\pi(c))/(n\pi(c))}$, which explodes as c grows: almost no draw lands in the tail. Shift the sampling distribution to $h = \phi(\cdot - c)$, i.e. $Y_i \sim N(c, 1)$. The weight is $\phi(y)/\phi(y-c) = e^{-cy+c^2/2}$, so

$$\hat{\pi}_{\text{imp}}(c) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{Y_i > c\}e^{-cY_i+c^2/2}.$$

Its second moment per draw is

$$e^{c^2}\int_c^\infty e^{-2cy}\phi(y-c)\,dy = e^{c^2}\int_c^\infty \phi(y+c)\,dy = e^{c^2}\Phi(-2c),$$

since $-2cy - \frac{1}{2}(y-c)^2 = -\frac{1}{2}(y+c)^2$. Hence $\operatorname{Var}\hat{\pi}_{\text{imp}}(c) = \frac{1}{n}\{e^{c^2}\Phi(-2c) - \pi(c)^2\}$. By Mills' ratio $\Phi(-t) \approx \phi(t)/t$ as $t \to \infty$, the squared relative error is about $c\sqrt{\pi/2}/n$, against $\sqrt{2\pi}\,ce^{c^2/2}/n$ for the naive estimator: the relative error grows like $c^{1/2}$ instead of $c^{1/2}e^{c^2/4}$.

10.3 Antithetic variates

If X and X' have the same distribution but are negatively correlated, then

$$\operatorname{Var}\Big[\frac{f(X)+f(X')}{2}\Big] = \frac{\operatorname{Var} f(X)}{2}(1+\rho), \qquad \rho = \operatorname{Corr}(f(X), f(X')),$$

which beats two independent draws when $\rho < 0$. For uniforms the natural pair is $(U, 1-U)$; for symmetric distributions about m, $(X, 2m - X)$.

Proposition 10.2

If f and g are both nondecreasing (or both nonincreasing), then $\operatorname{Cov}(f(X), g(X)) \geq 0$ for any random variable X for which the covariance exists. Consequently, for monotone h, $\operatorname{Cov}(h(U), h(1-U)) \leq 0$.

Proof. Let Y be an independent copy of X. Since f and g move in the same direction, $\{f(X) - f(Y)\}\{g(X) - g(Y)\} \geq 0$. Its expectation is

$$2\mathbb{E}f(X)g(X) - 2\mathbb{E}f(X)\mathbb{E}g(X) = 2\operatorname{Cov}(f(X), g(X)) \geq 0.$$

For the consequence, take $f = h$ and $g(u) = -h(1-u)$, which is monotone in the same direction as h. ■

Example 10.3

For $\mu = \mathbb{E}e^U = e - 1$: $\operatorname{Var} e^U = \frac{e^2-1}{2} - (e-1)^2 \approx 0.2420$ and $\operatorname{Cov}(e^U, e^{1-U}) = e - (e-1)^2 \approx -0.2342$. The antithetic average $\frac{1}{2}(e^U + e^{1-U})$ has variance $\frac{1}{2}(0.2420 - 0.2342) \approx 0.0039$, against $0.2420/2 = 0.1210$ for two independent uniforms: a reduction of about 97%.

10.4 Control variates

Suppose $Y = g(X)$ has a known mean μ_Y. For every c, $f(X) - c(Y - \mu_Y)$ is unbiased for μ, with variance

$$\operatorname{Var} f(X) + c^2 \operatorname{Var} Y - 2c \operatorname{Cov}(f(X), Y),$$

minimized at $c^\star = \operatorname{Cov}(f(X), Y)/\operatorname{Var} Y$, where it equals $\operatorname{Var} f(X)\,(1 - \rho^2)$, $\rho = \operatorname{Corr}(f(X), Y)$. The more correlated the control, the larger the gain. In practice $c^\star$ is estimated by the least-squares slope of $f(X_i)$ on Y_i (a small bias, negligible for large n).

Example 10.4

For $\mathbb{E}e^U$ with control $Y = U$ ($\mu_Y = \frac{1}{2}$, $\operatorname{Var} U = \frac{1}{12}$): $\operatorname{Cov}(e^U, U) = \mathbb{E}Ue^U - \frac{e-1}{2} = 1 - \frac{e-1}{2} \approx 0.1409$, so $c^\star = 12 \times 0.1409 \approx 1.6903$, and the variance per draw drops from 0.2420 to $0.2420 - 12(0.1409)^2 \approx 0.0039$.

10.5 A comparison: the size of a test

Example 10.5

With $X_1, \ldots, X_{25}$ i.i.d. Poisson(λ), the CLT test of $H_0 : \lambda = 2$ against $\lambda > 2$ rejects when $Z = (\bar{X} - 2)/\sqrt{2/25} > 1.645$, i.e. when $S = \sum X_i > c = \lfloor 25(2 + 1.645\sqrt{2/25}) \rfloor = 61$. Under λ, $S \sim \text{Poi}(25\lambda)$, so the size is $p = 1 - F_{50}(61) = 0.05568$, slightly above the nominal 0.05. Pretend we estimate p by simulation with N draws of S.

- *Standard*: $\hat{p} = \frac{1}{N}\sum \mathbf{1}\{S_j > c\}$ has variance $p(1-p)/N = 0.0526/N$.
- *Antithetic*: with $S_j = F_{50}^{-1}(U_j)$ and $S_j' = F_{50}^{-1}(1 - U_j)$, the two indicators cannot both be 1 (that would need $F_{50}(c) < U_j < 1 - F_{50}(c)$, impossible since $F_{50}(c) > \frac{1}{2}$), so their covariance is $-p^2$, and the average of N pairs has variance $p(1-2p)/(2N) = 0.0247/N$, compared with $p(1-p)/(2N) = 0.0263/N$ for $2N$ independent draws. That is only a 6% gain: the indicator of a rare event is a monotone function but nearly constant, so the negative correlation is weak.
- *Importance sampling*: draw $S_j \sim h = \text{Poi}(25\lambda')$ with $\lambda' = 2.4653$, the rejection threshold, so that about half the draws land in the rejection region. The weight is $w(s) = e^{-25(\lambda - \lambda')}(\lambda/\lambda')^s$, and the variance per draw is $\sum_{s>c} f_{50}(s)^2/h(s) - p^2 = 0.0063$, with f_{50} the Poi(50) pmf: 8.3 times smaller than standard Monte Carlo.

Each method gives a CLT confidence interval $\hat{p} \pm z_{1-\alpha/2}\sqrt{\widehat{\text{Var}}}$ with the variance estimated from the simulated summands. For the power at $\lambda \in [2.2, 4]$, the same code applies with F_{50} replaced by $F_{25\lambda}$. Once the rejection region is no longer rare ($\lambda \geq 2.5$, power ≥ 0.5), the advantage of the fixed envelope $\lambda' = 2.4653$ disappears. For large λ the weights $(\lambda/\lambda')^s$

become extremely variable and importance sampling becomes *worse* than standard Monte Carlo, whose variance $p(1-p)/N$ shrinks as the power approaches 1. Per draw, the importance sampling variance is 0.299 against 0.248 at $\lambda = 2.5$, and 17.3 against 0.053 at $\lambda = 3$. An envelope tuned to one λ is no good for all λ.

Lecture 11. Markov Chain Monte Carlo

Markov chain Monte Carlo (MCMC) simulates a distribution π by running a Markov chain whose limiting distribution is π, and averages along the path. The draws are dependent, but an ergodic theorem replaces the law of large numbers.

11.1 Markov chains

A discrete-time Markov chain on a countable state space S is a sequence $\{X_n\}$ with

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \ldots, X_0) = P(X_{n+1} = j \mid X_n = i) = P_{ij}.$$

We assume time homogeneity (P_{ij} does not depend on n). The rows of the **transition matrix** $P = (P_{ij})$ sum to one. Examples: the simple random walk on $\mathbb{Z}$ ($P_{i,i+1} = p$, $P_{i,i-1} = 1 - p$); gambler's ruin on $\{0, \ldots, N\}$ with absorbing states 0 and N.

Chapman–Kolmogorov. The n-step probabilities $P_{ij}^{(n)} = P(X_{m+n} = j \mid X_m = i)$ satisfy $P^{(m+n)} = P^{(m)}P^{(n)}$ (condition on the state at time m), so $P^{(n)} = P^n$. If X_0 has distribution $\pi^{(0)}$ (a row vector), X_n has $\pi^{(0)}P^n$.

Classification.

- j is *reachable* from i ($i \to j$) if $P_{ij}^{(n)} > 0$ for some $n \geq 0$; i and j *communicate* if $i \to j$ and $j \to i$. Communication classes partition S; the chain is **irreducible** if there is only one. The random walk is irreducible; gambler's ruin has classes $\{0\}$, $\{1, \ldots, N-1\}$, $\{N\}$.
- The **period** of i is $d(i) = \gcd\{n \geq 1 : P_{ii}^{(n)} > 0\}$; the chain is **aperiodic** if $d(i) = 1$ for all i.
- Let $\tau_{ii} = \min\{n \geq 1 : X_n = i\}$ given $X_0 = i$ (∞ if there is no return). State i is *recurrent* if $P(\tau_{ii} < \infty) = 1$, and *transient* otherwise; a recurrent state is **positive recurrent** if $\mathbb{E}\tau_{ii} < \infty$ and null recurrent otherwise. These are class properties.
- A chain is **ergodic** if it is aperiodic and positive recurrent. On a finite state space every state of an irreducible chain is positive recurrent, so irreducible plus aperiodic means ergodic.

Stationary distributions. π is **stationary** (invariant) if $\pi P = \pi$, $\pi \geq 0$, $\pi\mathbf{1} = 1$: started from π, the chain stays in π. If $P_{ij}^{(n)} \to \pi_j$ for all i, then π is stationary, since $\pi_k = \lim_n (P^{n+1})_{ik} = \sum_j \lim_n (P^n)_{ij} P_{jk} = (\pi P)_k$ (for finite S; in general by a dominated-convergence argument).

Theorem 11.1 (Limit theorems; stated without proof)

(a) An irreducible chain has a unique stationary distribution if and only if it is positive recurrent.

(b) If the chain is irreducible and ergodic, then $\lim_n P_{ij}^{(n)} = \pi_j$ exists independently of i, π is the unique stationary distribution, and $\pi_j = 1/\mathbb{E}\tau_{jj}$ is also the long-run fraction of time spent at j.

(c) If the chain is irreducible and aperiodic and a stationary distribution π exists, then $P_{ij}^{(n)} \to \pi_j$, and the chain is positive recurrent, hence ergodic.

Theorem 11.2 (Ergodic theorem)

If $\{X_n\}$ is irreducible and ergodic with stationary distribution π, and $\mathbb{E}_\pi|g(X)| < \infty$, then

$$\frac{1}{n}\sum_{k=1}^{n} g(X_k) \to \mathbb{E}_\pi g(X) \quad \text{almost surely, from any starting state.}$$

This extends the strong law from i.i.d. sequences to Markov chains and is the basis of MCMC: construct an irreducible, aperiodic, positive recurrent chain whose stationary distribution is the target f, run it, and average.

Detailed balance. The usual way to guarantee stationarity is the stronger **detailed balance** condition $\pi_i P_{ij} = \pi_j P_{ji}$ for all i, j.

Proposition 11.3

If π satisfies detailed balance, it is stationary. Moreover the stationary chain is *reversible*: $(X_0, \dots, X_n)$ and $(X_n, \dots, X_0)$ have the same distribution.

Proof. $(\pi P)_j = \sum_i \pi_i P_{ij} = \sum_i \pi_j P_{ji} = \pi_j \sum_i P_{ji} = \pi_j$. For reversibility, apply detailed balance repeatedly:

$$\pi_{i_0} P_{i_0 i_1} P_{i_1 i_2} \cdots P_{i_{n-1} i_n} = P_{i_1 i_0} \pi_{i_1} P_{i_1 i_2} \cdots = \cdots = \pi_{i_n} P_{i_n i_{n-1}} \cdots P_{i_1 i_0}.$$

■

General state spaces. On a continuous state space, P_{ij} becomes a **transition kernel** $K(x, y)$, the conditional density of X_{n+1} at y given $X_n = x$, with $\int K(x, y)\, dy = 1$ (allowing point masses). Stationarity reads $\pi(y) = \int \pi(x) K(x, y)\, dx$; detailed balance $\pi(x)K(x, y) = \pi(y)K(y, x)$ implies it by integrating over x. Irreducibility, recurrence and the limit theorems carry over with suitable definitions.

11.2 The Metropolis–Hastings algorithm

Motivation. For complicated high-dimensional posteriors, independent sampling is impossible. Metropolis–Hastings constructs a Markov chain whose stationary distribution is the target, using only density ratios, so normalizing constants are never needed.

Let f be the target density, known possibly only up to a constant, and $g(\cdot \mid x)$ a **proposal** density that is easy to sample. Start at $X^{(0)}$ with $f(X^{(0)}) > 0$. For $t = 0, 1, \dots$:

(1) draw a candidate $\tilde{X} \sim g(\cdot \mid X^{(t)})$;

(2) compute the **Metropolis–Hastings ratio**

$$R(X^{(t)}, \tilde{X}) = \frac{f(\tilde{X})\, g(X^{(t)} \mid \tilde{X})}{f(X^{(t)})\, g(\tilde{X} \mid X^{(t)})};$$

(3) set $X^{(t+1)} = \tilde{X}$ with probability $\alpha(X^{(t)}, \tilde{X}) = \min\{R, 1\}$, and $X^{(t+1)} = X^{(t)}$ otherwise.

Metropolis et al. (1953) used a symmetric proposal, $g(y \mid x) = g(x \mid y)$ (e.g. a random walk $y = x + \epsilon$), for which $R = f(\tilde{X})/f(X^{(t)})$: uphill moves are always accepted, downhill moves sometimes. Hastings (1970) allowed asymmetric g. Only ratios of f appear, so its normalizing constant is never needed.

Theorem 11.4

The Metropolis–Hastings chain satisfies detailed balance with respect to f; hence f is a stationary distribution.

Proof. The transition kernel is

$$K(x,y) = \alpha(x,y)g(y \mid x) + \{1 - r(x)\}\delta_x(y), \qquad r(x) = \int \alpha(x,y)g(y \mid x)\,dy,$$

where $r(x)$ is the probability of accepting a move from x and δ_x is a point mass at x. For the continuous part,

$$f(x)\alpha(x,y)g(y \mid x) = \min\{f(x)g(y \mid x),\, f(y)g(x \mid y)\} = f(y)\alpha(y,x)g(x \mid y),$$

since the middle expression is symmetric in (x,y). For the point mass, $f(x)\{1 - r(x)\}\delta_x(y)$ is nonzero only at $y = x$, where it equals $f(y)\{1 - r(y)\}\delta_y(x)$. Adding, $f(x)K(x,y) = f(y)K(y,x)$, and stationarity follows as in Proposition 11.3. ■

Stationarity is automatic; the other conditions of Theorem 11.2 depend on g and must be checked:

- *Irreducibility* holds if $f > 0$ on S and the chain with kernel $g(y \mid x)$ alone is irreducible.
- *Aperiodicity* usually holds because rejections give $P(X^{(t+1)} = X^{(t)}) > 0$. For instance, in a discrete chain with $g(i \mid i) > 0$ and, from each i, some proposable neighbour j with $f(j) < f(i)$, we get $\alpha(i,j) < 1$, so $r(i) < \sum_{j'} g(j' \mid i) = 1$ and $K(i,i) \geq 1 - r(i) > 0$, whence $d(i) = 1$.

Example 11.1 (Posterior sampling)

For a posterior $p(\theta \mid y) = c\,p(y \mid \theta)p(\theta)$ with unknown c, the ratio is

$$R(\theta, \tilde{\theta}) = \frac{p(y \mid \tilde{\theta})p(\tilde{\theta})g(\theta \mid \tilde{\theta})}{p(y \mid \theta)p(\theta)g(\tilde{\theta} \mid \theta)},$$

free of c. With the **independence chain** $g(\cdot \mid \theta) = p(\cdot)$ (propose from the prior), the prior cancels and R is the likelihood ratio $p(y \mid \tilde{\theta})/p(y \mid \theta)$.
For example, with $X \sim \mathrm{Bin}(n,p)$, $p = e^\theta/(1+e^\theta)$, and prior $\theta \sim N(0,\sigma^2)$ (the model of Example 8.2), the posterior is proportional to $e^{\theta x}(1+e^\theta)^{-n}e^{-\theta^2/(2\sigma^2)}$, whose normalizer has no closed form. Proposing $\tilde{\theta} \sim N(0,\sigma^2)$ and accepting with probability $\min\{1, e^{(\tilde{\theta}-\theta)x}\{(1+e^\theta)/(1+e^{\tilde{\theta}})\}^n\}$ samples from it.

11.3 The Gibbs sampler

For $X = (X_1, \ldots, X_p)$ with a product state space, write X_{-i} for all coordinates but i. The Gibbs sampler updates coordinates in turn from their **full conditionals**, which are often standard distributions:

$$X_i^{(t+1)} \sim f_{X_i \mid X_{-i}}(\cdot \mid X_1^{(t+1)}, \ldots, X_{i-1}^{(t+1)}, X_{i+1}^{(t)}, \ldots, X_p^{(t)}), \qquad i = 1, \ldots, p,$$

always conditioning on the latest values (as in Gauss–Seidel, Lecture 5).

Example 11.2 (Bivariate normal)

For $(X_1, X_2) \sim N\big(0, \left(\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right)\big)$ the conditionals are $X_1 \mid X_2 = x_2 \sim N(\rho x_2, 1-\rho^2)$ and symmetrically. Hence $X_1^{(t+1)} \sim N(\rho X_2^{(t)}, 1-\rho^2)$, then $X_2^{(t+1)} \sim N(\rho X_1^{(t+1)}, 1-\rho^2)$. The marginal

chain $X_2^{(t)}$ is autoregressive with coefficient ρ^2, so mixing is slow when $|\rho|$ is near 1.

Proposition 11.5

Each coordinate update of the Gibbs sampler is a Metropolis–Hastings step whose proposal is always accepted. Hence f is stationary for the Gibbs sampler.

Proof. Fix coordinate i and the current point x. The update proposes $\tilde{x}$ with $\tilde{x}_{-i} = x_{-i}$ and $\tilde{x}_i \sim f_{X_i|X_{-i}}(\cdot \mid x_{-i})$, i.e. the proposal density is $g_i(\tilde{x} \mid x) = f_{X_i|X_{-i}}(\tilde{x}_i \mid x_{-i})\,\delta(\tilde{x}_{-i} - x_{-i})$. On $\{\tilde{x}_{-i} = x_{-i}\}$, which has probability one, factor $f(x) = f_{X_{-i}}(x_{-i}) f_{X_i|X_{-i}}(x_i \mid x_{-i})$:

$$R(x, \tilde{x}) = \frac{f_{X_{-i}}(x_{-i}) f_{X_i|X_{-i}}(\tilde{x}_i \mid x_{-i}) \cdot f_{X_i|X_{-i}}(x_i \mid x_{-i})}{f_{X_{-i}}(x_{-i}) f_{X_i|X_{-i}}(x_i \mid x_{-i}) \cdot f_{X_i|X_{-i}}(\tilde{x}_i \mid x_{-i})} = 1.$$

By Theorem 11.4, each update leaves f invariant, and so does their composition. ■

So Gibbs is Metropolis–Hastings with a proposal that changes with the coordinate being updated (the composed chain is not reversible in general, but f is still stationary).

Example 11.3 (Ising model)

Place p particles around a circle (with $x_{p+1} = x_1$), each with spin $x_i \in \{-1, +1\}$, and let

$$p(x) = \frac{1}{Z} \exp\Big(\beta \sum_{j=1}^{p} x_j x_{j+1}\Big), \qquad \beta > 0,$$

which favours aligned neighbours. The normalizer Z is a sum of 2^p terms, infeasible for large p, but the full conditional of x_i depends only on its two neighbours and Z cancels. With $s_i = x_{i-1} + x_{i+1}$,

$$P(x_i = +1 \mid x_{-i}) = \frac{e^{\beta s_i}}{e^{\beta s_i} + e^{-\beta s_i}} = \frac{1}{1 + e^{-2\beta s_i}},$$

a logistic function of the neighbours. Equivalently, the current spin is flipped with probability $1/\{1 + \exp(2\beta x_i s_i)\}$.

11.4 Practical issues

- *Burn-in.* The chain starts away from stationarity; the initial portion of the run is discarded. How much is a judgement aided by trace plots and convergence diagnostics.
- *Mixing and stopping.* Strong autocorrelation (as in Example 11.2 with $|\rho| \approx 1$, or a random-walk proposal with a poorly chosen step size) means the chain explores slowly, and the effective sample size is much smaller than the number of iterations. The Monte Carlo standard error must account for the autocorrelation (e.g. by batch means).
- *Dependence.* To obtain nearly independent draws one can run N parallel chains from dispersed starting points and keep only the final values, or *thin* a single chain, keeping every k-th value $Y_m = X_{mk}$. Thinning reduces storage, not estimation error: for estimating $\mathbb{E}_\pi g$, the full chain is never worse.